

KRAKEN: Joint Optimization of Operator Placement and Communication for Distributed Stream Processing

Extended Technical Report

Ariane Ziehn
ariane.ziehn@tu-berlin.de
BIFOLD, TU Berlin
Germany

Finn Glück
f.glueck@campus.tu-berlin.de
TU Berlin
Germany

Philipp Emanuel Meran*
philippemanuel4@gmail.com
Ceteris AG
Germany

Steffen Zeuch
steffen.zeuch@tu-berlin.de
BIFOLD, TU Berlin
Germany

Volker Markl
volker.markl@tu-berlin.de
BIFOLD, TU Berlin, DFKI GmbH
Germany

Abstract

Internet of Things applications run continuous analytics over geo-distributed event streams and often rely on stateful binary operators, such as window joins and sequences. Efficient decentralized execution of these workloads depends on two tightly coupled decisions: where to execute operators (placement) and how to exchange events between nodes (communication). Most existing planners make these decisions sequentially by first fixing placement and then optimizing communication, which can miss globally better execution plans.

In this paper, we present KRAKEN, an execution planner that jointly optimizes both decisions by evaluating placement candidates under alternative communication schemes (all-push and push-pull) before committing to atomic deployment decisions. To compare different decisions, KRAKEN uses a bi-objective cost model that balances tuple volume and latency. Across diverse synthetic topologies and workloads, KRAKEN reduces tuple volume by 3× on average while lowering latency and planning time compared to the sequential baseline.

CCS Concepts

• Information systems → Stream management.

Keywords

fog-cloud environments, operator placement, communication optimization, decentralized query execution planning, state-space search

1 Introduction

Internet of Things (IoT) applications execute continuous queries over streams emitted by hundreds of geo-distributed sources [18, 45]. When these applications analyze relationships across multiple streams, queries often rely on *stateful binary operators*, such as window joins in general-purpose stream processing and sequences in complex event processing (CEP) [18, 20]. Two environments are commonly used to execute such workloads at scale: state-of-the-art

cloud-only solutions [9, 43], which suffer from high latency and backbone traffic, and early fog-only prototypes [1, 34], which offer heterogeneous resources but make it challenging to execute costly stateful operators at scale. To address these limitations, fog-cloud environments combine the centralized cloud with a decentralized fog layer for near-source compute [1, 12, 44]. The hierarchical topology of these unified environments leverages early data reduction while maintaining the cloud as a global sink and an always-on fallback with effectively unlimited compute resources [15, 38]. However, reducing data on its way to the cloud requires decentralized workload execution. In particular, this entails reasoning about both *operator placement*, i.e., deciding *where* each operator is executed, and *communication*, i.e., deciding *how* operator instances exchange data between nodes [4, 21, 25, 36]. Unary or stateless operators, such as filters and projections, can often be replicated and placed near their sources with little additional routing coordination to preserve correctness or avoid redundant traffic [6, 15, 45]. In contrast, stateful binary operators require multiple distributed input streams to reach a common execution node, and their intermediate results can grow substantially, in the worst case, exponentially [7, 31]. As a result, placement and communication are tightly coupled for these operators [6, 7, 18, 24, 37, 41].

This coupling creates a trade-off between where operators are placed and how their inputs are routed. Placing operators closer to sources can reduce the amount of data sent upward in the hierarchy [19, 21]. At the same time, each placement also determines which input and intermediate streams must be routed to the selected execution node, while the communication scheme determines whether these data are sent proactively or acquired selectively. The most common communication scheme is *all-push* [6, 7, 20], in which operators proactively forward all outputs to downstream operators and, ultimately, to the sink. All-push simplifies communication between nodes, but it can send high tuple volumes across bottlenecked backbone links, increasing queuing and end-to-end latency. Another communication scheme, proposed in the context of CEP, is *push-pull*. Push-pull communication can drastically reduce tuple volume by pushing low-rate streams and selectively pulling high-rate streams, but it requires additional buffering and increases latency due to pull round-trips [4, 17, 21, 36].

Most existing planners decide on placement and input acquisition sequentially. In particular, they either first fix operator placement

*Work performed while at TU Berlin.

This document is an extended author version of the paper published at DEBS '26, available at <https://doi.org/10.1145/3809481.3812615>.



This work is licensed under a Creative Commons Attribution 4.0 International License.

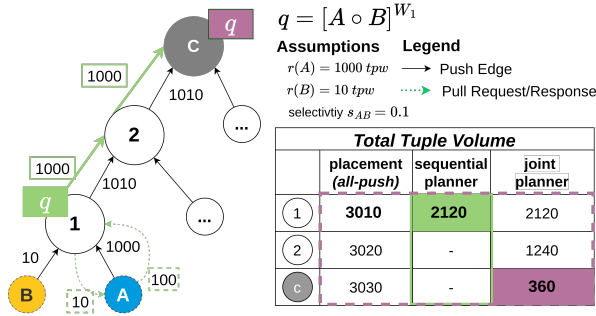


Figure 1: Sequential versus Joint Planning.

under all-push or assume centralized execution [6, 15], and then optimize input acquisition, i.e., which inputs are pushed or pulled and in which order, for this placement [4, 17, 36]. Because placement and communication are tightly coupled for stateful binary operators, this sequential planning can exclude globally better plans and motivates joint optimization of both decisions [2, 21, 23]. We make this coupling concrete with a simple industrial anomaly-detection example.

EXAMPLE 1. Consider the fragment of a fog-cloud topology in Figure 1, which shows a factory deployment where machine **A** generates a high-rate stream of vibration measurements, while a low-rate stream of manual fault confirmations originates from a control panel **B**. Tuples are forwarded from the sources via two intermediate fog nodes **1** and **2** to the cloud sink **C**. A continuous query q correlates recent readings from both streams over aligned time windows. A sequential planner first chooses a placement for q under all-push and therefore selects **1** as the cheapest placement because shipping the join output upward is cheaper than shipping both inputs over the remaining hops. It then optimizes the input acquisition for this fixed placement: tuples from **B** act as pull predicates, fetching only matching tuples from **A** ($s_{AB} = 0.1$). This pull step incurs a cost of 110 tuples by sending 10 predicate tuples from **B** to **A** and pulling back 100 matching tuples, instead of pushing all 1,000 tuples from **A**. Overall, this reduces tuple volume by 890 tuples, from 3,010 to 2,120 (highlighted in green). However, a globally better plan remains unreachable for the sequential planner. When placement and communication are optimized jointly by evaluating alternative placements together with multiple input-acquisition strategies, placing q in the cloud **C** under push-pull reduces the total tuple volume by almost 6 $\times$ (highlighted in purple), from 2,120 to 360.

The example highlights the limitation of sequential planning: fixing placement pre-commits key trade-offs and can rule out globally better decisions, such as placing q at **C**. To fully exploit decentralized fog resources in fog-cloud topologies, a planner must evaluate placement and input acquisition jointly before committing to a deployment decision, rather than optimizing them in separate stages. While exhaustive enumeration of all combinations is possible in principle, it does not scale to realistic IoT deployments with large topologies, where previous work reports runtimes of several hours already for 100 nodes [23, 38, 45]. This scalability barrier has a theoretical basis. Both decision problems are computationally hard, even

in isolation, i.e., operator placement is NP-hard [7, 41], and optimization of communication is NP-complete [36]. Consequently, the joint problem of simultaneously deciding both is at least as hard.

In this paper, we introduce KRAKEN, an execution planner that *jointly* optimizes placement and communication for stateful IoT workloads in fog-cloud environments. Given a query workload and latency constraints, KRAKEN explores the joint space of placement and input acquisition along the data paths to the cloud sink, guided by a capacity-aware bi-objective cost model that captures tuple volume and transmission latency. By reasoning about both deployment decisions together, KRAKEN identifies execution plans that sequential planners cannot reach. We implement KRAKEN in a simulation environment and evaluate it against the sequential approach as our primary baseline and three additional baselines, comparing tuple volume, latency, and planning time. Across synthetic fog-cloud topologies and diverse workloads, KRAKEN reduces tuple volume by about 3 $\times$ on average, while also reducing the median transmission latency by 14% and median planning time by 7 $\times$ compared to the sequential approach. In sum, we make the following contributions:

- We formulate the joint problem of operator placement and communication for stateful IoT workloads in hierarchical, heterogeneous fog-cloud environments (Section 3).
- We present KRAKEN, a joint planner that casts the problem as a state-space search over *atomic deployment decisions*, and evaluates candidates via a bi-objective cost model while respecting compute and latency constraints (Section 4).
- We evaluate KRAKEN against four baselines, showing that joint planning reduces tuple volume while meeting latency constraints and remaining computationally tractable (Section 5).

We discuss related work in Section 6 and conclude in Section 7.

2 PRELIMINARIES

In the following, we first define the system and workload model in Section 2.1. Building on this, we introduce our plan, rate, and cost notation in Section 2.2. Finally, we introduce the core decisions in deployment planning for stream processing in Section 2.3.

2.1 System and Workload Model

Logical and physical streams. A stream S is a continuous, unbounded sequence of timestamped tuples, or events, with a common schema [5, 8, 26, 30, 39]. For a tuple $t \in S$, we write $t.ts$ for its timestamp. In IoT environments, geo-distributed devices, such as sensors, produce *physical* streams S_k^P , where $k \in \mathbb{N}^+$ [20, 45]. Physical streams with the same context, e.g., temperature, form one *logical* stream S . We write $S^P(S)$ for the non-empty collection of physical streams assigned to the logical stream S and ϵ for the universe of logical streams. We distinguish physical and logical streams only where needed to model distributed sources and routing costs. Otherwise, planning operates over logical and intermediate streams.

Fog-cloud topology. We model the hierarchical topology of fog-cloud environments as a directed acyclic graph $\Gamma = (N, E)$ with nodes N and edges $E \subseteq N \times N$ defining data flows to the cloud [45]. While edges are directed, as depicted in Figure 1, the underlying physical links are bidirectional for control flow (e.g., pull requests).

Nodes. Each node n has a per-window compute budget $\text{cpu} : N \rightarrow \mathbb{R}_{\geq 0}$. Nodes are partitioned into three disjoint roles: $N = N^+ \cup N^f \cup N^-$. *Sources* $N^+ \subseteq N$ are leaf nodes that inject physical streams. A source function $\text{src} : \epsilon \rightarrow 2^{N^+}$ assigns to each logical stream $S \in \epsilon$ the source nodes that produce its physical streams S_j^P (e.g., $\text{src}(A) = \{ \text{A}, \dots \}$). For sources, we assume $\text{cpu}(n) = 0$ (no processing). *Workers* $N^f \subseteq N$ have finite $\text{cpu}(n)$ and thus can transmit and process tuples [1, 45]. *Sinks* $N^- \subseteq N$ represent result consumers. We refer to a single cloud *root* $n^- \in N^-$, such as C , as the central sink with effectively unbounded capacity. It serves as a centralized fallback for operators that cannot be placed in the fog or are not beneficial to place there. Conceptually, a set of sinks is also possible. To reflect the fog-cloud resource hierarchies, we instantiate $\text{cpu}(n)$ with increasing budgets, from low-end edge devices through mid-end fog nodes toward the scalable cloud sink n^- [1, 45].

Edges. Each directed edge $e = (x \rightarrow y) \in E$ with $x, y \in N$ represents a network link with a positive weight $\ell(e) \in \mathbb{R}^+$, used as a proxy for transmission latency and per-tuple transfer cost [17]. We write $\text{dist}(x, y)$ for the shortest-path distance from x to y in Γ under ℓ , and set $\text{dist}(x, y) = \infty$ if no path exists. Unless stated otherwise, we use hop-count distances as a neutral structural default by setting $\ell(e) = 1$ for all $e \in E$. Deployment-specific penalties, e.g., for hierarchy layers, can be modeled by assigning larger edge weights $\ell(e)$.

Query language and operators. A workload $Q = \{q_1, \dots, q_m\}$ is a finite set of continuous queries. Each query $q = (O, \text{in})$ is an ordered operator DAG, where O is the set of operators and $\text{in} : O \rightarrow O^{\leq 2}$ is a partial function that assigns each operator $o \in O$ to its input operators [5, 7, 10, 13, 36]. Source operators ingest logical streams, internal operators transform streams, and sink operators consume final result streams [5, 13]. Operator outputs form intermediate logical streams that may serve as inputs to downstream operators. Let ϵ^{int} denote the set of intermediate streams produced during evaluation of Q , and let $\epsilon^Q := \epsilon \cup \epsilon^{\text{int}}$ be the set of all logical streams that occur in Q .

We focus on stateful *binary* stream operators $o = (A, B, \Theta, W)$, where A and B denote the two input streams, Θ is a (possibly empty) set of predicates, and W is a window operator [3, 7, 42, 46]. Although different window types exist, we focus on time-based window operators, where each operator W discretizes both input streams into a time-aligned sequence of windows $\text{win}(W)$ before evaluating o [40]. Predicates Θ constrain the event pairs that contribute to the operator's output.

Semantics. Given two logical input streams A and B , a set of aligned windows produced by W , and predicates Θ , a binary operator produces a new (intermediate) logical stream AB defined as

$$AB := \bigcup_{w \in \text{win}(W)} \{ \{a \oplus b \mid a \in A, b \in B, a.ts, b.ts \in w \wedge \forall \theta \in \Theta : \theta(a, b)\} \},$$

where $\cup$ denotes multiset union and $\oplus$ schema concatenation.

2.2 Query Execution Plan and Cost Notation

An execution plan π for a workload Q on Γ consists of a placement map $\varphi : O_Q \rightarrow N$ from operators to nodes, where $O_Q := \bigcup_{q \in Q} O_q$. Routing is fixed to shortest paths under $\text{dist}(\cdot, \cdot)$.

Rate model. Each physical stream S_j^P has a rate $r(S_j^P) \in \mathbb{N}^+$, i.e., tuples per window $w \in \text{win}(W)$. Unary predicates (per-stream filters) are assumed to be applied at the sources and are absorbed into

$r(S_j^P)$. Accordingly, the predicate set Θ contains only *multi-stream* conditions, i.e., join predicates or temporal order constraints. The global per-window rate of a logical stream S is $r(S) = \sum_{S_j^P \in S^P(S)} r(S_j^P)$. The *local* per-window rate of (sub-)stream S is $r^n(S)$, i.e., the sum of all physical stream rates of S delivered to node $n \in N$ under plan π . For a binary stateful operator o with logical inputs A and B executed at node x , we define the input rate $r_{\text{in}}^x(o)$ and, under the standard independence assumption, model the per-window output rate $r_{\text{out}}^x(o)$ as the number of tuple pairs scaled by selectivity $s_\Theta \in [0, 1]$ [7]:

$$r_{\text{in}}^x(o) = r^x(A) + r^x(B), \quad r_{\text{out}}^x(o) = s_\Theta r^x(A) r^x(B).$$

We assume fixed source rates $r(\cdot)$, identical for all $S_j^P \in S^P(S)$, and selectivities s_Θ during planning, obtained, e.g., via profiling or statistics.

Cost metrics. A workload Q admits multiple plans, where each plan π is assigned a cost, often dominated by the tuple volume [4, 7, 36]. We write $\text{cost}(\pi)$ for the cost of a plan π . In the following, we introduce the metrics of our bi-objective cost model, i.e., tuple volume $V(\cdot)$ and latency $L(\cdot)$. Both metrics are refined in Section 4.3 at the granularity of individual steps in the acquisition sequence.

Tuple volume. We measure tuple volume as the *number of transmitted tuples per window*. To treat all-push and push-pull uniformly, we count pushed tuples, pull requests, and pull replies as tuples using the respective stream rates [36]. For a plan π , let $T_e(\pi)$ denote the number of tuples transmitted over edge e . Then, the total tuple volume of π is

$$V(\pi) = \sum_{e \in E} T_e(\pi).$$

Transmission latency. We measure transmission latency as the longest communication path induced by a plan. Let $\mathcal{F}(\pi) \subseteq N \times N$ denote the node pairs between which plan π induces a data transfer, and let $1[(x, y)_{\text{pull}}]$ indicate whether this transfer is realized via push or pull. Assuming symmetric distances, we define

$$L(\pi) = \max_{(x, y) \in \mathcal{F}(\pi)} (1 + 1[(x, y)_{\text{pull}}]) \text{dist}(x, y).$$

$L(\pi)$ upper-bounds the critical communication delay implied by π . Admissible plans must satisfy the user-defined bound $L(\pi) \leq L_{\max}$.

EXAMPLE 2. Consider our topology Γ and query q on logical streams A and B as shown in Figure 1. In a centralized all-push deployment, q runs entirely in the cloud C and all sources forward their tuples to it. With one A -source with rate $r(A) = 1,000$ and one B -source with rate $r(B) = 10$, each sent over three edges, the total tuple volume is $V = 3 \cdot (1000 + 10) = 3030$ tuples and the hop-based latency is $L = 3$.

Because realistic selectivities are often low, i.e., s_Θ typically falls in $[10^{-6}, 10^{-1}]$, decentralized execution can avoid forwarding irrelevant tuples across the topology [4, 6, 42, 46].

2.3 Deployment Planning

To enable decentralized execution, distributed stream processing systems require deployment strategies that decide (i) how to decompose the workload, (ii) where to place each subquery (placement), (iii) how inputs are acquired and routed through the network (acquisition modes and sequences), and (iv) how to traverse the search space guided by a cost function [2, 7, 21, 22, 24, 35].

Decomposition. INEV [6] is a state-of-the-art placement approach with a multi-query decomposition technique. Using this technique, we split the workload Q into subqueries (projections) and

derive a dependency-respecting processing order. Let $Q' = (P, \text{pred})$ denote the resulting decomposition, where P is the set of *subqueries* and $\text{pred} : P \rightarrow 2^P$ encodes a precedence relation. In particular, for each $p \in P$, $\text{pred}(p)$ is the set of subqueries whose outputs are required as inputs to p . INEV's technique assumes all-push communication, under which input rates and CPU demands are highest. Thus, it provides a conservative upper bound for node capacity checks. In the remainder, we plan at the granularity of subqueries $p \in P$ as units of deployment. Each subquery p is itself a valid query over logical input streams $\epsilon_p \subseteq \epsilon_Q$, where ϵ_p denotes the streams required to execute p .

Placement. A baseline strategy is *single-node placement*, which assigns each subquery p to one strategic execution node $n = \varphi(p)$. For a subquery p with a binary stateful operator o over logical inputs A and B , node n must be able to receive both inputs. Let $\text{prov}(S) \subseteq N$ denote the set of provider nodes for logical stream S . For logical streams $S \in \epsilon$, providers coincide with sources, i.e., $\text{prov}(S) = \text{src}(S)$. For intermediate inputs $S \in \epsilon^{\text{int}}$, providers are the execution nodes that produce S in the current plan. The placement at n is feasible iff $\text{dist}(n^+, n) < \infty$ for all $S \in \{A, B\}$ and all $n^+ \in \text{prov}(S)$, and n has sufficient compute capacity to host p . In the example of Figure 1, all physical substreams of A and B first converge at ①, making it a feasible execution node. Under all-push and ignoring capacity and latency constraints, placing a binary operator o at $x \in N^f$ is typically beneficial if it is rate-reducing, i.e., $r_{\text{in}}^x(o) > r_{\text{out}}^x(o)$, since this shrinks the tuple volume forwarded upstream. Conversely, if $r_{\text{in}}^x(o) < r_{\text{out}}^x(o)$, o fans out and is placed at the sink(s), where the expanded output is consumed.

An advanced strategy is *multi-node placement*, which replicates the computation of a subquery across multiple nodes instead of executing it at a single site. While multi-node placement is straightforward for unary operators (e.g., filters or projections), it is non-trivial for stateful binary operators because their outputs are compositions of tuples from two inputs and can grow quickly [7, 31]. One multi-node placement approach for stateful binary CEP operators [7] exists that considers decentralized environments with partial visibility. At a high level, it relaxes the single-node requirement, which necessitates routing both inputs to one execution node, by requiring only *one* logical input $S \in \{A, B\}$ at node $x \in N^f$, i.e., x receives all physical substreams of S under the plan. Multi-node placement can then replicate a logical operator across multiple such nodes, where each node x can compute local results by combining this complete logical input stream with a disjoint subset of the physical streams of the other input. Thus, smaller intermediate results, rather than the entire other input, are shipped to the cloud. If no feasible or beneficial node exists in the topology, multi-node placement degenerates to single-node placement at the first convergence point of both inputs.

Communication. Given a placement, *acquisition sequences* determine how tuples and intermediate results are exchanged between nodes to provide the inputs of each deployed subquery [20]. PrePP [36] is a state-of-the-art approach for optimizing input acquisition under predicate-based push-pull. In contrast to traditional push-pull [4, 17, 23], it can restrict returned tuples by time constraints and predicates. Using this model, we represent the input acquisition of subquery p at node n as an *ordered acquisition sequence* $d_{\text{steps}} = \langle a_1, \dots, a_k \rangle$. Each step $a_i = (\Psi_i, \mathcal{R}_i)$ consists of (i) a *pull set* $\Psi_i \subseteq \epsilon^Q$ of logical streams that are available at n (under the current partial plan), and (ii) a *request set* $\mathcal{R}_i \subseteq \epsilon^P$ of logical streams whose

tuples are acquired from providers $\text{prov}(\cdot)$. If $\Psi_i = \emptyset$, then a_i is a *push step*, where providers forward all tuples for the relevant windows to n . If $\Psi_i \neq \emptyset$, then a_i is a *pull step*, where tuples from Ψ_i act as predicates to retrieve only matching tuples from streams in $\mathcal{R}_i$. In pull steps, providers additionally buffer the requested streams (typically at least the content of the last two windows [4]).

All-push consists of single push steps, whereas *push-pull* mixes push and pull steps on a per-input choice to trade tuple volume for memory and round-trip latency [4, 17, 36]. For a binary operator o , push-pull offers three canonical acquisition choices: (i) push both inputs in one step, $\langle \langle \emptyset, \{A, B\} \rangle \rangle$; (ii) push A and then pull B , $\langle \langle \emptyset, \{A\} \rangle, \langle \{A\}, \{B\} \rangle \rangle$; or (iii) push B and then pull A , $\langle \langle \emptyset, \{B\} \rangle, \langle \{B\}, \{A\} \rangle \rangle$. For subqueries with multiple inputs, the space of acquisition sequences grows combinatorially as each input may be pushed or pulled, and pull requests can be issued in different orders [36]. Finer-grained pulls typically reduce tuple volume but increase latency due to additional round-trips.

3 Joint Optimization Problem

Efficient decentralized execution requires deciding both *where* subqueries run and *how* they acquire their inputs. We are given a hierarchical fog-cloud topology $\Gamma = (N, E)$ with per-node compute capacities $\text{cpu}(n)$, a workload decomposition $Q' = (P, \text{pred})$ over logical streams ϵ^Q , and a latency bound L_{max} . A decentralized execution plan π assigns each subquery $p \in P$ (i) an execution node $\varphi(p) \in N$ and (ii) an acquisition sequence $d_{\text{steps}} = \langle a_1, \dots, a_k \rangle$ consisting of push and pull steps. We evaluate the quality of π by a bi-objective cost that captures the tuple volume $V(\pi)$, defined as the number of transmitted tuples per window (counting push tuples, pull requests, and pull replies), and the transmission latency $L(\pi)$:

$$\text{cost}(\pi) = (V(\pi), L(\pi)).$$

We require admissible plans to satisfy per-node compute capacities and the latency bound:

$$\begin{aligned} (1) \quad & \sum_{p \in P: \varphi(p) = n} \kappa_p \cdot r_{\text{in}}^n(p | \pi) \leq \text{cpu}(n) \quad \forall n \in N, \\ (2) \quad & L(\pi) \leq L_{\text{max}}. \end{aligned}$$

Here, κ_p is the processing cost per input event of subquery p , and $\kappa_p \cdot r_{\text{in}}^n(p | \pi)$ is its per-window CPU demand when executed at node n under π . We formalize the problem of joint placement and acquisition planning as follows:

PROBLEM 1. *Given a topology Γ , a workload decomposition $Q' = (P, \text{pred})$, and a latency bound L_{max} , compute an admissible plan π by jointly deciding, for each subquery $p \in P$, an execution node $\varphi(p) \in N$ and an acquisition sequence d_{steps} under consideration of different communication schemes, to minimize $\text{cost}(\pi)$ over admissible plans.*

We address Problem 1 with KRAKEN, a joint planner for hierarchical fog-cloud topologies.

4 The KRAKEN planner

In this section, we introduce KRAKEN, a planner that jointly optimizes placement and communication along the paths to the cloud sink using state-space search. We present KRAKEN's state-space model in Section 4.1 and describe its main components and planning workflow in Section 4.2. In Section 4.3, we present our ingress-egress cost

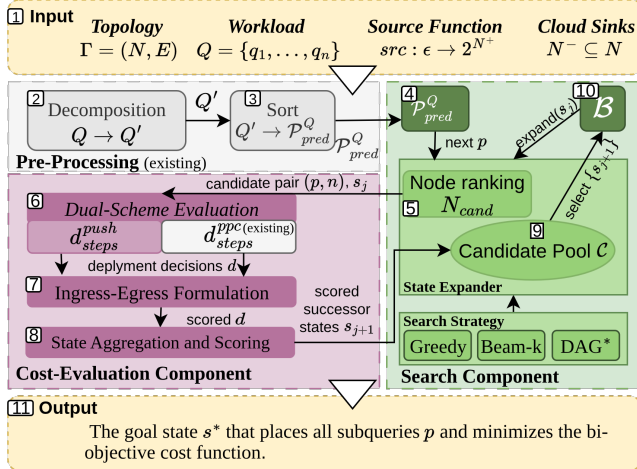


Figure 2: KRAKEN Overview.

model for scoring deployment decisions based on the cost of input acquisition and result forwarding. Then, we describe the implemented search strategies in Section 4.4, discuss correctness and complexity in Section 4.5, and clarify scope and extensibility in Section 4.6. Finally, we discuss how KRAKEN’s single-node placement assumption can be relaxed through predicate-aware multi-node placement, establishing a formal basis for future extensions beyond the evaluated planner in Section 4.7.

4.1 State-Space Model

We formalize KRAKEN’s joint optimization as a state-space search, where each state represents a partial or complete execution plan. We first define a *deployment decision* that captures an atomic configuration for one subquery. We then introduce the *state representation* used by the search.

Deployment decision. We define a deployment decision as a tuple $d = (p, n, d_{\text{steps}})$, where $p \in P$ is a subquery, $n \in N$ is its execution node, and d_{steps} is the acquisition sequence executed at n to obtain all inputs required by p . Compared to an INEv deployment unit, a KRAKEN deployment decision replaces the abstract forwarding sub-graph with PrePP’s concrete acquisition sequence d_{steps} [6, 36]. This representation binds placement and communication into a single deployment decision, allowing joint optimization.

State representation. We define a state as a tuple $s = (\mathcal{D}_s, \mathcal{T}_s)$, where $\mathcal{D}_s : P' \rightarrow \mathcal{D}$ maps placed subqueries $P' \subseteq P$ to their deployment decisions, and $\mathcal{T}_s : N \rightarrow 2^{\epsilon^Q}$ tracks which streams and intermediate results are available at each node. When a subquery is placed at a node, $\mathcal{T}_s$ is updated to reflect the acquired inputs and produced outputs. This allows subsequent placements to exploit locally available data without incurring additional ingress costs (Section 4.3). The search space $\mathcal{S}$ is the set of states reachable from the initial state s_0 , i.e., an empty plan in which no subqueries are placed.

4.2 Components and Planning Workflow

Figure 2 summarizes the components of KRAKEN and their interaction. We model joint planning over the inputs in step $\boxed{1}$, i.e., a topology

Γ , workload Q , source function src , and sink set N^- . KRAKEN preprocesses Q using INEv [6] to obtain a precedence-respecting sequence of subqueries $\mathcal{P}_{pred}^Q$ (gray box, step [2]–[3]). This order ensures that, when a subquery is processed, all predecessor subqueries that produce the required intermediate inputs have already been placed. Candidate placements are then restricted to nodes that can access the required source or the provider nodes of intermediate inputs.

For its joint search, KRAKEN leverages two main components: the *search component* (green box, detailed in Section 4.4), which incrementally constructs execution plans, and the *cost-evaluation component* (purple box, detailed in Section 4.3), which evaluates candidate deployment decisions under our unified cost model. The search component maintains a frontier $\mathcal{B}$ of states s_j (step [10]). To expand a state s_j , the State Expander selects the next unplaced subquery p from $\mathcal{P}_{pred}^Q$ (step [4]). In step [5], it identifies feasible candidate nodes $N_{\text{cand}} \subseteq N^f \cup N^-$ that can receive all required inputs and have sufficient compute capacity. It then ranks these nodes by decreasing input availability in $\mathcal{T}_{s_j}$ and increasing hop-based distance to the input providers. For each subquery-node pair (p, n) with $n \in N_{\text{cand}}$, the cost-evaluation component performs *dual-scheme evaluation* (step [6]) to derive acquisition sequences under two communication schemes: (i) an all-push sequence $d_{\text{steps}}^{\text{push}}$ and (ii) a tuple-volume-greedy push-pull sequence $d_{\text{steps}}^{\text{ppc}}$ with the predicate-based push-pull planner PrePP [36]. Since candidate nodes are ordered by increasing hop-based distance, once an all-push sequence violates the latency bound L_{max} , all subsequent nodes can be pruned, and candidate exploration for p terminates. Step [6] yields two deployment decisions that jointly fix where subqueries run and how they acquire their inputs. When $d_{\text{steps}}^{\text{ppc}}$ coincides with $d_{\text{steps}}^{\text{push}}$, KRAKEN retains only a single deployment decision. Next, deployment decisions are scored using our *ingress-egress formulation* in step [7] and combined with s_j to produce successor states s_{j+1} in step [8]. All successor states are inserted into the *candidate pool* $\mathcal{C}$ (step [9]), from which the Search Strategy selects the next frontier $\mathcal{B}$ (step [10], Section 4.4). Thus, each expansion step places exactly one subquery while branching over alternative placement and communication choices. The planning process repeats steps [4]–[10] until all subqueries are placed. Then, KRAKEN returns the goal state s^* , i.e., a complete execution plan with the smallest scalar cost $F(s^*)$ (step [11], Section 4.3.2).

In sum, KRAKEN’s state-space search constructs a complete execution plan π as a sequence of joint deployment decisions, one per subquery. We illustrate KRAKEN’s state-space search for our motivating example from Section 1 in Figure 3, detailing one expand iteration.

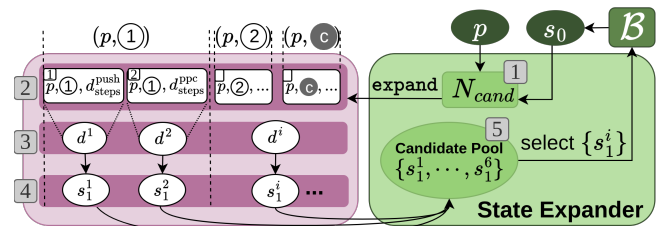


Figure 3: One expand iteration in KRAKEN’s state-space search.

EXAMPLE 3. Our example contains a single subquery, i.e., $\mathcal{P}_\delta^Q = \langle p \rangle$, where $p = [A \circ B]^{W_1}$. Thus, the State Expander is invoked on the initial state $s_0 = (\mathcal{D}_{s_0}, \mathcal{T}_{s_0})$ with $\mathcal{D}_{s_0} = \emptyset$, i.e., no subqueries have been placed yet, and $\mathcal{T}_{s_0}$ only reflects the stream availability at the sources. It selects the unplaced subquery p and creates an ordered candidate list N_{cand} [1]. Here, no node violates L_{max} under distance-based ordering, hence $N_{\text{cand}} = \langle \textcircled{1}, \textcircled{2}, \textcircled{3} \rangle$. For each pair (p, n) , the cost-evaluation component triggers the dual-scheme evaluation [2] which creates the two deployment decisions $(p, n, d_{\text{steps}}^{\text{push}})$ for acquisition with all-push and $(p, n, d_{\text{steps}}^{\text{ppc}})$ for acquisition with push-pull. Each decision is scored using our ingress-egress formulation [3] and combined with s_0 [4], yielding a total of six successor states $s_1^1, \dots, s_1^6$. Each successor state $s_1^i = (\mathcal{D}_{s_1^i}, \mathcal{T}_{s_1^i})$ updates $\mathcal{D}_{s_0}$ by mapping p to the chosen decision and updates $\mathcal{T}_{s_0}$ at the selected node to reflect the acquired inputs and the output stream of p . For instance, s_1^1 that uses all-push to compute p at $\textcircled{1}$, now has the streams A , B , and AB available at this node. All successor states are inserted into the candidate pool of the State Expander [5].

4.3 Unified Cost Model

In the following, we present KRAKEN's unified cost model, which scores and aggregates each deployment decision during state expansion (Figure 2, steps [6]–[8]). In Section 4.3.1, we first introduce our topology-aware *ingress-egress formulation*, which captures the cost of acquiring the required input streams and forwarding result streams toward the sink(s). Then, we describe how KRAKEN combines deployment decisions with the current state and scalarizes the resulting bi-objective cost into a single score $F(\cdot)$ for ranking in Section 4.3.2.

4.3.1 Ingress-Egress Formulation. To evaluate a deployment decision d under the current state s , our cost model assigns (i) a tuple volume V_d and (ii) a latency L_d . The tuple volume V_d decomposes into an ingress and an egress component:

$$V_d = V_{\text{ing}}(d, s) + V_{\text{egr}}(d).$$

The latency L_d is defined recursively and captures the critical path for producing the output of p at n . It combines local processing latency ℓ_d^p , ingress transmission latency $\ell_{\text{ing}}^t(d, s)$, egress transmission latency $\ell_{\text{egr}}^t(d)$ and the maximum latency of predecessor subqueries $\text{pred}(p)$:

$$L_d = \xi \cdot \ell_d^p + \overbrace{(\ell_{\text{ing}}^t(d, s) + \ell_{\text{egr}}^t(d))}^{\ell_d^t} + \max_{u \in \text{pred}(p)} L_{\mathcal{D}_s(u)},$$

where $\max_\emptyset = 0$ and $\xi \in \{0, 1\}$ (set $\xi = 0$ to ignore ℓ_d^p). In the following, we detail how the different components are derived.

Ingress. Ingress costs quantify the execution of an acquisition sequence d_{steps} , where each step $a_i = (\Psi_i, \mathcal{R}_i)$ satisfies $\Psi_i \subseteq \mathcal{T}_s(n)$ and $\mathcal{R}_i \subseteq \epsilon^p$. The *ingress tuple volume* is the sum of the volume of requests V_{req} and responses V_{res} in all acquisition steps in d_{steps} :

$$V_{\text{ing}}(d, s) = \sum_{a_i \in d_{\text{steps}}} (V_{\text{req}}(a_i, s) + V_{\text{res}}(a_i, s)).$$

To quantify the volumes of requests and responses, we first identify the input providers of each stream in the state s . For a stream $S \in \epsilon^Q$, let $\text{prov}_s(S)$ denote the set of nodes at which S is available:

$$\text{prov}_s(S) = \{n^+ \in N \mid S \in \mathcal{T}_s(n^+)\}.$$

To account for locally available streams that induce no further ingress cost, we define *remote providers* N_i^{rem} as the set of nodes that still must transmit a physical or logical stream to n in step a_i .

$$N_i^{\text{rem}} = \bigcup_{S \in \mathcal{R}_i} N_i^{\text{rem}}(S) \quad \text{with} \quad N_i^{\text{rem}}(S) = \begin{cases} \emptyset & \text{if } S \in \mathcal{T}_s(n), \\ \text{prov}_s(S) \setminus \{n\} & \text{otherwise.} \end{cases}$$

By construction, $N_i^{\text{rem}}(S) = \emptyset$ if $S \in \mathcal{T}_s(n)$, i.e., S is locally available at n and no network acquisition is needed. Otherwise, $N_i^{\text{rem}}(S)$ lists the remote providers from which n must acquire tuples of S in step a_i . Accordingly, the request volume $V_{\text{req}}(a_i, s)$ accounts for transmitting predicate tuples from streams $S_j \in \Psi_i$ at node n to the remote providers contacted in step a_i to acquire the streams in $\mathcal{R}_i$:

$$V_{\text{req}}(a_i, s) = \left(\sum_{S_j \in \Psi_i} r(S_j) \cdot \sigma(\Psi_i) \right) \cdot \left(\sum_{n^+ \in N_i^{\text{rem}}} \text{dist}(n, n^+) \right).$$

Here, $r(\cdot)$ is the global per-window rate, and $\sigma(\Psi_i)$ represents the *current* cumulative selectivity of the streams in Ψ_i based on the steps executed so far, taking into account filtering in previous acquisition steps. If $\Psi_i = \emptyset$, the request cost is 0, and a_i is a push step.

$V_{\text{res}}(a_i, s)$ accounts for transmitting the requested matching tuples of streams $S_k \in \mathcal{R}_i$ from these remote providers back to n .

$$V_{\text{res}}(a_i, s) = \sum_{S_k \in \mathcal{R}_i} \sum_{n^+ \in N_i^{\text{rem}}(S_k)} (r^{n^+}(S_k) \cdot \sigma(S_k \mid \Psi_i) \cdot \text{dist}(n^+, n)),$$

where $r^{n^+}(\cdot)$ is the local stream rate at node n^+ , and $\sigma(S \mid \Psi_i)$ denotes the selectivity of applying predicates from Ψ_i to S . When $\Psi_i = \emptyset$, then $\sigma(S \mid \Psi_i) = 1$, yielding the push cost for events in $\mathcal{R}_i$.

For *transmission latency*, ℓ_{ing}^t sums the per-step transmission time to the farthest contacted provider. A push step ($\Psi_i = \emptyset$) incurs only a response leg ($n^+ \rightarrow n$), whereas a pull step additionally incurs a request leg ($n \rightarrow n^+$):

$$\ell_{\text{ing}}^t(d, s) = \sum_{a_i \in d_{\text{steps}}} \left(\max_{n^+ \in N_i^{\text{rem}}} \text{dist}(n^+, n) + 1[\Psi_i \neq \emptyset] \cdot \max_{n^+ \in N_i^{\text{rem}}} \text{dist}(n, n^+) \right),$$

where $1[\cdot]$ indicates pull steps and $\max_\emptyset(\cdot) = 0$.

Relation to push-pull cost modeling. Our ingress formulation extends PrePP's [36] acquisition model from centralized queries to decentralized subqueries in hierarchical topologies. By instantiating providers through $\text{prov}_s(\cdot)$ and weighting requests and responses by shortest-path distance $\text{dist}(\cdot, \cdot)$, it becomes topology-aware and rewards locality by making already available inputs free of costs.

Egress. Egress costs quantify the dissemination of final outputs from p 's execution node n to the sink(s). Let $P_{\text{out}} \subseteq P$ be subqueries whose outputs are sink-consumed, and let $N_{\text{rem}}^-(n) = N^- \setminus \{n\}$ denote the sinks not co-located with n . Then, the *egress tuple volume* is

$$V_{\text{egr}}(d) = \begin{cases} r_{\text{out}}^n(p) \sum_{n^- \in N_{\text{rem}}^-(n)} \text{dist}(n, n^-) & \text{if } p \in P_{\text{out}}, \\ 0 & \text{otherwise,} \end{cases}$$

where $r_{\text{out}}^n(p)$ is the output rate of p . The *egress latency* $\ell_{\text{egr}}^t(d)$ is the time required to reach the furthest sink (with $\max_\emptyset(0) = 0$) and is evaluated for every deployment decision to ensure compliance with L_{max} .

$$\ell_{\text{egr}}^t(d) = \max_{n^- \in N_{\text{rem}}^-(n)} \text{dist}(n, n^-).$$

Fixed sinks and explicit result forwarding. Unlike INEv [6], which assumes the final consumer is co-located with the evaluation node,

our setting fixes the sink set N^- . We therefore model the cost of disseminating final outputs explicitly. Omitting this term can bias placements toward fog nodes, even when forwarding the (potentially high-rate) final result to distant sinks dominates the overall cost.

4.3.2 State Aggregation and Scoring. Each successor state s_{j+1} aggregates the bi-objective cost of s_j and d by adding the tuple volumes and updating latency as the current critical path (with $V_{s_0} = 0$ and $L_{s_0} = 0$):

$$V_{s_{j+1}} = V_{s_j} + V_d \quad \text{and} \quad L_{s_{j+1}} = \max(L_{s_j}, L_d).$$

If $L_{s_{j+1}} > L_{\max}$, s_{j+1} is discarded. To guide the search over the candidate pool, KRAKEN ranks states in ascending order using a scalar objective $F(s)$ that aggregates tuple volume and latency. We compute $F(s)$ by min-max normalizing both metrics over the current candidate pool C for dimensional consistency and apply a weighted sum:

$$F(s) = \alpha \cdot \frac{V_s - V_C^{\min}}{V_C^{\max} - V_C^{\min}} + (1 - \alpha) \cdot \frac{L_s - L_C^{\min}}{L_C^{\max} - L_C^{\min}}.$$

where $V_C^{\min}$, $V_C^{\max}$ and $L_C^{\min}$, $L_C^{\max}$ are the minimum and maximum values among the current candidate pool C . If $V_C^{\max} = V_C^{\min}$ (resp. $L_C^{\max} = L_C^{\min}$), we set the corresponding normalized term to 0. The weight parameter $\alpha \in [0, 1]$ controls the trade-off between tuple volume and latency. Since both metrics are conflicting objectives, improvements in one often degrade the other [4, 21].

Recall our motivating example from Section 1 and the expansion step in Figure 3. This step generates six deployment decisions for the single-subquery workload $p = [A \circ B]^{W_1}$ and inserts the corresponding successor states into the candidate pool of the search component.

EXAMPLE 4. We detail the cost computation for $d_4 = (p, \textcircled{2}, d_{\text{steps}}^{\text{ppc}})$, where $d_{\text{steps}}^{\text{ppc}} = \langle a_1, a_2 \rangle$ with $a_1 = (\emptyset, \{B\})$ and $a_2 = (\{B\}, \{A\})$.

Tuple volume. Step a_1 is a push step for stream B ($\Psi_1 = \emptyset$), thus

$$V_{\text{req}}(a_1, s_0) = 0 \quad \text{and} \quad V_{\text{res}}(a_1, s_0) = r(B) \cdot \text{dist}(B, \textcircled{2}) = 10 \cdot 2 = 20.$$

Step a_2 uses tuples of B as pull predicates to request tuples from A . The request volume V_{req} and the response volume V_{res} are

$$V_{\text{req}}(a_2, s_0) = r(B) \cdot \text{dist}(\textcircled{2}, A) = 10 \cdot 2 = 20,$$

$$V_{\text{res}}(a_2, s_0) = r(A) \cdot \sigma(A | B) \cdot \text{dist}(A, \textcircled{2}) = 1000 \cdot 0.1 \cdot 2 = 200.$$

Thus, $V_{\text{ing}}(d_4, s_0) = (0 + 20) + (20 + 200) = 240$. Since $p \in P_{\text{out}}$, the result must be disseminated to the sink $\textcircled{c}$. Assuming $r_{\text{out}}^n(p) = 1000$ and $\text{dist}(\textcircled{2}, \textcircled{c}) = 1$, we obtain $V_{\text{egr}}(d_4) = 1000$ and the total tuple volume is $V_{d_4} = V_{\text{ing}}(d_4, s_0) + V_{\text{egr}}(d_4) = 1240$.

Latency. We set $\xi = 0$ and assume there are no predecessors. For ingress latency, a_1 is a push step ($\Psi_1 = \emptyset$) and a_2 is a pull step ($\Psi_2 = \{B\}$). The farthest contacted provider in a_1 and a_2 is at distance 2, hence

$$\ell_{\text{ing}}^t(d_4, s_0) = \underbrace{(2 + 1[\emptyset \neq \emptyset] \cdot 2)}_{a_1} + \underbrace{(2 + 1[\{B\} \neq \emptyset] \cdot 2)}_{a_2} = (2 + 0) + (2 + 2) = 6.$$

With $\ell_{\text{egr}}^t(d_4) = \text{dist}(\textcircled{2}, \textcircled{c}) = 1$, we obtain $L_{d_4} = \ell_{\text{ing}}^t(d_4, s_0) + \ell_{\text{egr}}^t(d_4) = 7$.

In this first expand step, the objectives of the successor state coincide with the decision ones, e.g., d_4 induces s_1^4 with $V_{s_1^4} = V_{d_4}$ and $L_{s_1^4} = L_{d_4}$. Finally, the search ranks the successor states using the scalar score $F(s)$. Using the tuple volumes reported in Figure 1 and the remaining latencies: $L_{d_1} = L_{d_3} = L_{d_5} = 3$, $L_{d_2} = 5$, $L_{d_6} = 9$, the ranking depends on the trade-off parameter α . With $\alpha = 1$ (volume-only), the best candidate is s_1^6 with $V_{d_6} = 360 = V_C^{\min}$. With $\alpha = 0$ (latency-only), the best

candidate is s_1^1 with $L = 3$, and among all states with equivalent latency, the lowest tuple volume. Feasibility constraints can prune states before scoring, e.g., if $L_{\max} \leq 8$, then d_6 with $L_{d_6} = 9$ is excluded and d_4 becomes top-ranked for intermediate weights (i.e., $\alpha \geq 0.6$). Since the workload contains one subquery, the selected successor state of this expand step is also a goal state s^* .

Summary. For each feasible candidate placement (p, n) , KRAKEN evaluates all-push and push-pull acquisition. For each resulting deployment decision $d = (p, n, d_{\text{steps}})$, our model computes the ingress and egress costs for tuple volume and latency, resulting in the bi-objective cost (V_d, L_d) . To expand a state s_j , KRAKEN generates one successor s_{j+1} per deployment decision, aggregates the resulting costs at the state level, and ranks successors using $F(\cdot)$. We next describe KRAKEN's search component, which explores the resulting state space.

4.4 Search Component

The KRAKEN search component iteratively expands the current frontier while enforcing feasibility constraints and selects the next frontier based on the objective ranking and chosen search strategy (Figure 2, steps [9]–[10]). In particular, we implement three strategies that trade planning quality for computational efficiency, i.e., greedy, beam- k , and DAG* search. Greedy selects the best successor state at each step and is commonly used for computationally hard deployment problems such as placement and input acquisition [6, 23, 36], often in combination with additional heuristics. It is computationally and memory efficient, but can converge to a locally optimal plan [23]. In contrast, beam- k retains the top- k successor states (the *beam*) to improve plan quality at a higher computational cost. This increases the chance of finding a globally superior plan without resorting to exhaustive enumeration, at the cost of higher overhead that typically grows with k . When the search terminates, it returns the goal state s^* with the smallest score $F(s^*)$ among all generated candidates. DAG* [38] is an A*-style graph search that prioritizes partial plans by their estimated total cost. This estimate combines the already realized cost of the partial plan with an admissible lower-bound heuristic for the remaining unplanned subqueries. For each unplaced subquery, DAG* derives this heuristic by summing the minimum ingress cost for each required input stream over all legally reachable placement nodes. If the subquery produces a final output, the heuristic additionally includes the minimum egress cost to the nearest sink. Since each term is minimized independently, the resulting heuristic cannot overestimate the remaining cost and is therefore admissible.

4.5 Correctness and Guarantees

We first analyze the computational complexity of Problem 1 (joint placement and acquisition planning) and summarize the correctness and feasibility properties of plans produced by KRAKEN.

THEOREM 1 (JOINT PLANNING IS NP-HARD). Problem 1 is NP-hard.

Proof (sketch). KRAKEN combines classical operator placement in distributed environments [6, 7, 41] with the problem of selecting the acquisition mode (push or pull) per input stream [36]. We argue NP-hardness by restriction. If communication is fixed to all-push (i.e., no acquisition-mode choices), Problem 1 reduces to classical

operator placement on non-tree networks, which is NP-hard [7, 41]. Since our joint problem generalizes this special case, it is NP-hard.

For completeness, Problem 1 also subsumes acquisition-mode selection. When placement is fixed, the remaining decision is to choose acquisition modes for the input streams on the upstream subgraph rooted at the placement node, which coincides with the PrePP problem and is NP-complete [36].

Feasibility, search-space size, and optimality. KRAKEN enforces feasibility by pruning during expansion: the State Expander discards any deployment decision and successor state that violates per-node compute capacities or exceeds the latency bound $L_{\max}$ under our latency model. Since only feasible successors are retained, any returned goal state s^* (and its corresponding plan π) satisfies the capacity constraints per node and $L(\pi) \leq L_{\max}$.

If each subquery $p \in P$ has at most m feasible placement candidates and both communication schemes (all-push and push-pull) are admissible per candidate, the joint search space contains $O(m^{|P|})$ placement candidates and $O((2m)^{|P|})$ deployment decisions. Because KRAKEN relies on search-space restrictions introduced by its currently integrated components, i.e., INEV's fixed subquery decomposition and the selection of a single PrePP acquisition sequence per placement candidate, none of the implemented search strategies, including DAG*, guarantees globally optimal plans [6, 23, 36, 38].

Semantic correctness. KRAKEN preserves query semantics because it reuses INEV's semantics-preserving decomposition and dependency order [6, 7], and does not alter operator logic. In particular, it *decides* where subqueries execute and how inputs are acquired. For push-pull decisions, correctness follows from the correctness of PrePP's input acquisition [36]. KRAKEN composes these steps through semantics-preserving state transitions.

4.6 Scope, Adaptivity, and Extensibility

In this section, we clarify the scope and assumptions underlying KRAKEN and discuss limitations and possible extensions.

Placement assumption. KRAKEN uses single-node placement, i.e., it places each subquery at one node where, for each required logical input stream S , all physical streams in $S^P(S)$ are available. Multi-node placement (or horizontal parallelism) is common in homogeneous cloud clusters, where shared infrastructure provides global stream visibility and operators can be replicated over system-defined data partitions. In contrast, we focus on the attached fog layer of fog-cloud topologies, where topology-induced partial stream visibility makes horizontal scaling non-trivial because correctness depends on feasible routing decisions [14]. Since horizontal scaling in the cloud is orthogonal to our focus, we abstract the cloud as a single root node.

Multi-node placement is a natural extension of KRAKEN, but it is not merely a matter of replicating operators across arbitrary fog nodes. The existing multi-node placement approach [7] relaxes the requirement of single-node placement, but still ties applicability and correctness to strong visibility assumptions, i.e., a candidate node must receive at least one complete logical input stream. However, in hierarchical fog-cloud topologies with geographically dispersed sources and directed dataflow, streams from different branches often converge only near the cloud. As a result, such visibility requirements can force stateful binary operators toward higher-level nodes and may degenerate to single-node placement, failing to exploit lower

fog nodes that would otherwise have sufficient computational capacity. Our findings show that this requirement can be weakened when predicates restrict which physical substreams can jointly contribute to an output tuple, e.g., through region- or key-based constraints that are naturally present in IoT workloads [28, 42]. In this case, applicability and correctness do not require visibility over an *entire* logical input stream, but only over those *pairs of physical substreams* that are compatible under the predicate. This observation enables predicate-aware multi-node placement, where stateful binary operators are decomposed into smaller local instances, whose union is equivalent to centralized evaluation. Integrating this extension into KRAKEN would require the planner to select a set of placement decisions instead of a single deployment node per subquery, and to combine these decisions with routing and communication-mode selection. We therefore treat multi-node placement as an orthogonal extension to the single-node placement model studied in the main part of this paper and discuss this direction in Section 4.7.

Mitigation strategies for prolonged planning times. Due to the large optimization space, KRAKEN's planning can become computationally expensive, e.g., placement grows with network size and push-pull optimization with query length (Section 5). In practice, planning time can be controlled through mitigation strategies that trade optimality for tractability, e.g., timeouts that place remaining subqueries in the cloud and only optimize push-pull communication. These mechanisms can be integrated into the current framework.

Adaptivity. We evaluate KRAKEN in a static planning setting where the topology, workload, input rates, and selectivities are fixed at planning time. In dynamic deployments, these inputs can change over time [34, 38], requiring adaptive mechanisms that trigger re-planning, ideally restricted to affected subqueries or topology branches [14]. KRAKEN is not inherently limited to static planning and can be extended with such mechanisms. For instance, when rate or selectivity changes make a fog placement no longer beneficial, the conservative all-push baseline for feasible placement selection and the cloud fallback provide safe alternatives, such as reverting push-pull acquisition to all-push or migrating subqueries toward the cloud with push-pull optimization. Integrating adaptivity is future work.

Extensibility. Existing planners often couple deployment concerns into a monolithic optimization procedure. In contrast, KRAKEN exposes decomposition, placement, communication, cost evaluation, and search traversal through explicit interfaces decoupled from specific implementations. This separation enables the study of different component combinations under controlled synthetic topologies and workloads. First, alternative decomposition strategies beyond INEV are possible, including network-aware decompositions tailored to restricted hierarchical topologies. Such decompositions must still provide a dependency order over subqueries, since KRAKEN incrementally places subqueries along the fog-cloud hierarchy. Second, the cost model can be refined from structural hop-based latency to time-based latency or extended with additional metrics such as link utilization or memory consumption. Third, our acquisition model can be extended to a more exhaustive variant that retains multiple acquisition sequences per placement candidate as distinct deployment decisions, increasing the branching factor and planning time. Additionally, alternative acquisition models beyond PrePP, e.g., Comet [17], can be incorporated by providing acquisition sequences for a given placement. Fourth, alternative search strategies beyond our selection, such

as depth-first search [15] or branch-and-bound, can be integrated into the state-space search to systematically evaluate their benefits and trade-offs across different workloads and topology settings.

Beyond the planner itself, the simulation environment exposes configurable topology and workload parameters, including topology size, depth, connectivity, and workload characteristics (Section 5.1). These parameters support reproducibility and enable extensions with richer topology and workload models, as well as realistic latency weights and resource constraints for follow-up studies.

4.7 Distributivity of Stateful Binary Operators and Predicate-Aware Multi-Node Placement

Section 4.6 identifies multi-node placement as an orthogonal extension of KRAKEN’s single-node placement model. In this section, we make this extension precise for stateful binary operators. To this end, we first show that stateful binary operators admit a distributivity property that enables multi-node placement in general, and then introduce predicate-aware multi-node placement as a conceptual extension of KRAKEN.

THEOREM 2 (DISTRIBUTIVITY OF STATEFUL BINARY OPERATORS). *Let $[A \circ_{\Theta} B]^W$ be a stateful binary operator with logical inputs A and B , predicate set Θ , and window operator W . Let $S^P(A)$ and $S^P(B)$ denote the physical substreams of A and B , respectively. Assume that, for every tuple pair contributing to $[A \circ_{\Theta} B]^W$, both window membership under W and predicate evaluation under Θ can be decided from the corresponding pair of physical substreams (A_i^P, B_j^P) without requiring tuples from any other physical substream. Then*

$$[A \circ_{\Theta} B]^W = \bigcup_{A_i^P \in S^P(A)} \bigcup_{B_j^P \in S^P(B)} [A_i^P \circ_{\Theta} B_j^P]^W.$$

Proof. Since the physical substreams in $S^P(A)$ and $S^P(B)$ are pairwise disjoint, every tuple pair contributing to the global join result belongs to exactly one pair of physical substreams (A_i^P, B_j^P) . By assumption, both window membership under W and predicate evaluation under Θ can be decided locally for each such pair without requiring tuples from other physical substreams. Hence, every tuple pair contributing to the global operator result is evaluated by exactly one local operator instance $[A_i^P \circ_{\Theta} B_j^P]^W$, i.e., the one induced by the unique physical substreams containing its two tuples. Conversely, every tuple pair emitted by a local operator instance is also valid in the global evaluation. Thus, the global join result equals the multiset union of the local join results over all pairs of physical substreams.

Theorem 2 does not depend on a specific window type per se, but on whether window membership and predicate evaluation can be decided locally for physical substream pairs. For time-based window types with fixed clock-aligned boundaries, such as tumbling and sliding windows, this condition is typically satisfied directly [11, 27]. For context-sensitive window types, such as session window joins or interval joins, distributivity requires stronger visibility conditions than for fixed clock-aligned windows, as tuple values determine the window creation [11, 48].

Predicate-aware multi-node placement. Based on this distributivity property, we extend existing multi-node placement with a predicate-aware placement criterion for pairwise-local stateful binary operator semantics.

COROLLARY (PREDICATE-RESTRICTED DISTRIBUTIVITY). *Let $\theta \in \Theta$ induce a logical partitioning of the inputs A and B into partition sets*

$$\mathcal{P}_A = \{A_1^P, \dots, A_m^P\}, \quad \mathcal{P}_B = \{B_1^P, \dots, B_n^P\},$$

Further, let

$$\mathcal{M}_{\theta} \subseteq \mathcal{P}_A \times \mathcal{P}_B$$

denote the set of θ -compatible partition pairs, i.e., those pairs (A_i^P, B_j^P) for which tuples from A_i^P may join with tuples from B_j^P under θ . If Theorem 2 applies, then the global join can be restricted to these compatible partition pairs:

$$[A \circ_{\Theta} B]^W \equiv \bigcup_{(A_i^P, B_j^P) \in \mathcal{M}_{\theta}} [A_i^P \circ_{\Theta} B_j^P]^W.$$

We refer to the resulting weaker locality condition as θ -complete visibility: a node $n \in N^f$ has θ -complete visibility for $[A \circ_{\Theta} B]^W$ if it receives all predicate-compatible inputs required to evaluate its assigned local stateful binary operator instance correctly. For time-based window types, this typically means one complete predicate-induced logical partition of one input side together with a subset of the required θ -compatible partitions from the other side. Again, for context-sensitive window types, the locality requirement is stronger. The node must receive the complete predicate-induced logical partitions of both inputs needed to determine session boundaries locally. Hence, local correctness depends only on the visibility of the required compatible partition pairs, not on the complete visibility of an entire logical input stream. Globally, the planner must ensure that every pair in $\mathcal{M}_{\theta}$ is covered by exactly one operator instance. Thus, the Corollary enables *predicate-aware multi-node placement (θ -MNP)*, where nodes evaluate a smaller subtask of the global operator, i.e., their assigned local operator instances.

Correctness. Correctness follows from the distributivity over θ -compatible partition pairs, together with the complete coverage of these pairs.

THEOREM 3 (CORRECTNESS OF θ -MNP). *Assume that: (i) Theorem 2 applies to $[A \circ_{\Theta} B]^W$, (ii) every θ -compatible pair in $\mathcal{M}_{\theta}$ is evaluated by exactly one operator instance assigned to a node, and (iii) emitted results are validated against the full predicate set Θ before output. Then the union of all local results produced under θ -MNP is sound and complete with respect to the centralized evaluation of $[A \circ_{\Theta} B]^W$.*

Proof (sketch). Soundness follows because each local operator instance evaluates only tuple pairs that satisfy the local window condition and because emitted results are validated against the full predicate set Θ . Completeness follows from the Corollary, which reduces the global evaluation to the multiset union over all θ -compatible partition pairs in $\mathcal{M}_{\theta}$. Assumption (ii) guarantees that every such pair is evaluated by at least one local instance, while the *exactly once* requirement prevents duplicates under multiset semantics.

EXAMPLE 5. Consider the topology Γ with workload Q in Figure 4. Assume that streams A_1 and B_1 belong to region 1, streams A_2 and B_2 belong to region 2, and streams A_3 , A_4 and B_3 belong to region 3. Let the join predicate θ_{AB} restrict matches to streams from the same region, and let the resulting join selectivity be 0.1. Then only same-region partition pairs belong to $\mathcal{M}_{\theta_{AB}}$. Accordingly, nodes ①,

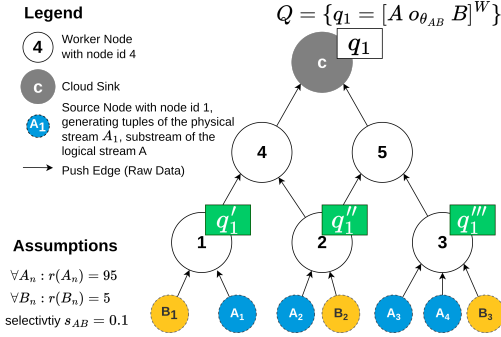


Figure 4: A topology Γ with workload Q and θ -MNP.

(2), and (3) each have θ -complete visibility for exactly their regional subproblem. While conventional multi-node placement would place q_1 only at the cloud sink (c), θ -MNP replicates the stateful binary operator across these nodes. Each node evaluates only its assigned compatible pairs, e.g., node (1) evaluates $q'_1 = [A_1 \circ_{\theta} B_1]^W$, node (2) evaluates $q''_1 = [A_2 \circ_{\theta} B_2]^W$, and node (3) evaluates $q'''_1 = [A_3 \circ_{\theta} B_3]^W \cup [A_4 \circ_{\theta} B_3]^W$. No cross-region join instance is required because such pairs are excluded by θ_{AB} . Accordingly, nodes (1), (2), and (3) compute their local operator results and forward only intermediate tuples to the sink (c). As a result, the total tuple volume is reduced from 1,185 to 775 compared to the centralized baseline.

Discussion and outlook. We establish the formal foundation for predicate-aware multi-node placement of stateful binary operators. They show that the execution of a global stateful binary operator can be decomposed along predicate-compatible physical substream pairs and evaluated across multiple fog nodes, provided that every compatible pair is covered by exactly one local operator instance. This extends the single-node placement model used by KRAKEN without changing the operator semantics. Integrating θ -MNP into KRAKEN’s planning procedure remains future work. Such an integration would require the planner to reason about predicate-induced logical partitions rather than complete logical streams. Consequently, the placement decision for a subquery would no longer consist of selecting one deployment decision, but of selecting a set of local operator instances (or replicas) whose combined execution covers the global operator. This extension increases the search space, because each replica requires a joint deployment decision, where each decision may interact differently with routing and communication-mode choices. The number of replicas depends on the partitions induced by the predicate and grows with the number of predicate-compatible partition pairs. At the same time, predicate-aware decomposition can reduce tuple volume by pushing stateful processing closer to the sources and avoiding transmissions for predicate-incompatible stream pairs. Combining θ -MNP with DAG*-style search is therefore a promising direction for extending KRAKEN beyond single-node placement while preserving correctness under partial stream visibility.

5 Evaluation

In this section, we evaluate KRAKEN. First, we describe our setup and simulation environment in Section 5.1. Then, we compare KRAKEN against four baselines in Section 5.2. Finally, we examine the impact

of KRAKEN’s search strategies, bi-objective trade-offs, and heterogeneous resource constraints in Section 5.3.

5.1 Setup

We evaluate KRAKEN in an offline simulator written in Python and derived from the INEv codebase, which allows us to compare all approaches under identical inputs and cost assumptions. The simulator generates (i) synthetic hierarchical topologies with per-node compute capacities, unit-weight links (hop-based distances), and a fixed cloud sink, and (ii) synthetic query workloads with fixed rates and selectivities. For each topology-workload instance, we run the planners considered in the respective experiment, obtain their execution plans, and evaluate them under our cost model. All experiments run on a server equipped with a 16-core CPU and 47 GiB of RAM.

Baselines. We compare KRAKEN against four approaches:

Centralized. A baseline that places the entire workload at the cloud sink under all-push and serves as the 1.0 $\times$ normalization target.

*INEv** [6]. A tuple-volume-greedy in-network placement planner that assumes the all-push scheme. This baseline is based on INEv but adapted to our fixed-sink setting by adding explicit result forwarding costs (Section 4.3.1). Without this adaptation, INEv can underestimate plans that place subqueries with high-rate final outputs in-network, which in our simulator led to plans whose evaluated tuple volume was up to 33 $\times$ higher than the centralized baseline.

PrePP [36]. A tuple-volume-greedy communication optimizer that assumes the entire workload is placed at the cloud sink and optimizes acquisition sequences under the push-pull scheme.

OP $\rightarrow$ PPC. A sequential baseline that first computes placements using INEv* and then applies PrePP to optimize acquisition sequences under the push-pull scheme for the resulting fixed placements.

Topology and workload parameters. We generate synthetic hierarchical network topologies and query workloads using Zipfian distributions [6]. Following prior work [6, 36], we conduct seven *parameter studies* over comparable or extended ranges: (1) *Network size* N is the total number of nodes in the network. The depth of the network (number of layers) grows approximately as $\log_2(N)$. For example, for $N = 1,000$, the depth is 10. (2) *Parent factor* controls network connectivity by defining the maximum number of parents per node as a relative factor f of the number of layers L . For example, with $L = 3$ and $f = 1.8$, a node has at most $\lceil 1.8 \cdot 3 \rceil = 6$ parents. (3) *Workload size* specifies the number of queries in the workload, and (4) *query length* determines the average number of binary operators per query. (5) *Number of streams* defines the number of distinct streams in the workload, and (6) *event skew* controls rate heterogeneity across streams (larger values correspond to less skew and 2.0 yields nearly

Table 1: Experimental Parameters.

Parameter	Default	Values
(1) Network Size	100	[10,30,50,100,200,500,1000]
(2) Parent Factor	1.8	[1.0,1.2,1.4,1.6,1.8,2.0]
(3) Workload Size	5	[3,5,7,10,20,40]
(4) Query Length	5	[3,5,8,10]
(5) Number of Streams	6	[4,6,8,10,20]
(6) Event Skew	2.0	[1.0,1.2,1.4,1.6,1.8,2.0]
(7) Node Event Ratio	0.7	[0.1,0.3,0.5,0.7,0.9]

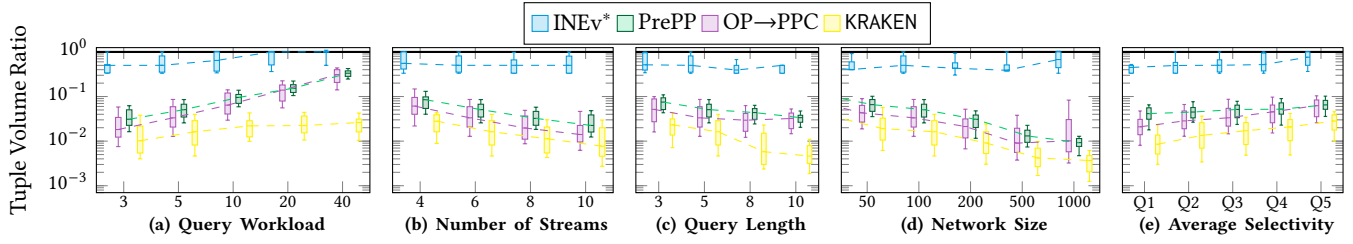


Figure 5: Comparison of KRAKEN against State-of-the-Art Approaches on Tuple Volume.

uniform rates). (7) The *node event ratio* defines the probability with which each source node generates events of one of the streams. In addition, we classify the runs post hoc by *average selectivity* and report the results using selectivity quantiles based on the observed average predicate selectivity (as a percentage): $Q_1 = [0, 20]$, $Q_2 = [20, 40]$, $Q_3 = [40, 60]$, $Q_4 = [60, 80]$, and $Q_5 = [80, 100]$.

Although the evaluation is synthetic, the parameters in Table 1 capture key characteristics of fog-cloud deployments, i.e., network scale and hierarchy depth (1), connectivity (2), workload size and query complexity (3-4), stream and rate heterogeneity (5-6), and source distribution (7). Thus, the parameter studies test whether KRAKEN’s gains persist across various topology-workload instances and indicate where additional heuristics or migration strategies may be beneficial. To this end, each study varies one topology or workload parameter while keeping all other parameters at their default values. We execute each configuration at least 50 times.

Metrics and search strategy. To assess the quality of different execution plans, we measure tuple volume and latency normalized to the *centralized baseline* [6, 24, 36]. The *tuple volume ratio* [6, 36] is the ratio between the tuple volume of a distributed plan and that of the centralized baseline. The *hop-latency ratio* is the ratio between the hop-based transmission latency induced by a distributed plan and that of the centralized baseline. Additionally, we compare the wall-clock planning time per planner. For all metrics, lower values indicate better performance. Unless stated otherwise, we run KRAKEN with greedy search and set $\alpha = 1$ for baseline comparisons, so that KRAKEN optimizes tuple volume only and matches the objective used by the baselines.

5.2 KRAKEN versus baselines

In this section, we compare KRAKEN against four baselines: Centralized (all-push), INEv* (all-push), PrePP, and OP→PPC. Thus, KRAKEN is evaluated under greedy search with $\alpha = 1$. Since the baselines do not model latency constraints or heterogeneous compute capacities, we run all planners under a simplified setting with homogeneous nodes, unlimited CPU capacity, and no global latency bound. This ensures that observed differences stem from planning decisions rather than from optimizing different objectives or constraints.

We first analyze the tuple volume in Section 5.2.1, then study the transmission latency in Section 5.2.2 and the planning time in Section 5.2.3. We consider the seven parameters presented in Table 1 and execute all configurations for each metric. For readability, we show plots for the most informative parameters and omit others per metric when their curves mirror the same trend. All summary statistics still reflect the full set of runs across all parameters.

5.2.1 Tuple Volume Comparison. In this experiment, we investigate the impact of different parameter settings on the tuple volume.

Observations. We present results in Figure 5, plotted on a logarithmic scale, with the centralized baseline shown as a black horizontal line in all subfigures. We make three observations. First, across almost all parameter settings, the boxplot medians follow a consistent ordering: $INEv^* > PrePP > OP \rightarrow PPC > KRAKEN$. Second, all planners exhibit similar *qualitative* trends: their median values increase or decrease in the same direction when a parameter changes. For example, as the number of nodes increases (Figure 5d), all planners reduce tuple volume. Third, KRAKEN shows a larger spread (longer whiskers) than the other planners in several settings, indicating stronger instance-to-instance variability. The omitted plots for parent factor, node event ratio, and event skew preserve the same median ordering. In our simulator, all three show only minor or nearly constant trends.

Results. The median ordering in Figure 5 is also reflected in the aggregate ratios across all runs: the median tuple volume ratio drops from 0.50 (INEv*) to 0.05 (PrePP), 0.03 (OP→PPC), and 0.016 (KRAKEN). This ordering aligns with the decision scope of each approach. INEv* is a pure placement planner under the all-push scheme and therefore yields the highest tuple volume. PrePP reduces tuple volume through optimized push-pull acquisition, but assumes a centralized placement at the sink. OP→PPC combines both ideas sequentially by first fixing a placement with INEv* and then optimizing acquisition with PrePP. In contrast, KRAKEN jointly optimizes placement and acquisition, exploring plan combinations that OP→PPC cannot reach. A larger spread of KRAKEN is expected. Depending on the topology-workload instance (e.g., available locality and predicate selectivities), joint optimization can either unlock very low-volume plans or yield only modest gains, which widens the distribution. Overall, KRAKEN outperforms OP→PPC in 88.7% of the runs by an average factor of $3\times$ (min. $1.8\times$, max. $11.8\times$). Compared to PrePP, KRAKEN is superior in 90.3% of the runs with an average factor of $4.6\times$ (min. $1.9\times$, max. $12\times$). Compared to INEv*, KRAKEN is superior in all runs with an average factor of $45\times$ (min. $7\times$, max. $183\times$).

5.2.2 Hop Latency Comparison. In this experiment, we investigate the impact of different parameter settings on the hop latency.

Observations. We present our results in Figure 6. INEv* coincides with the centralized baseline and appears as a black horizontal line in all subfigures. We make three observations. First, across all parameter settings, the medians follow a consistent ordering: $PrePP > OP \rightarrow PPC > KRAKEN$. Second, PrePP is most sensitive to challenging workloads, such as many or long queries, and many streams (Figures 6a-c), where its median latency ratio increases sharply. OP→PPC and

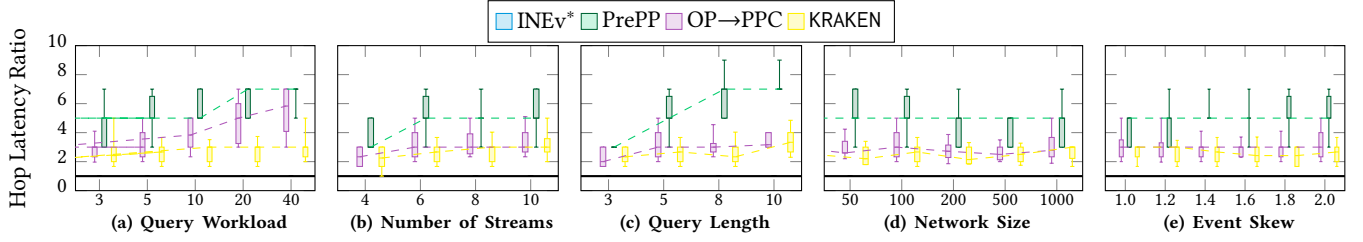


Figure 6: Comparison of KRAKEN against State-of-the-Art Approaches on Hop Latency Ratio.

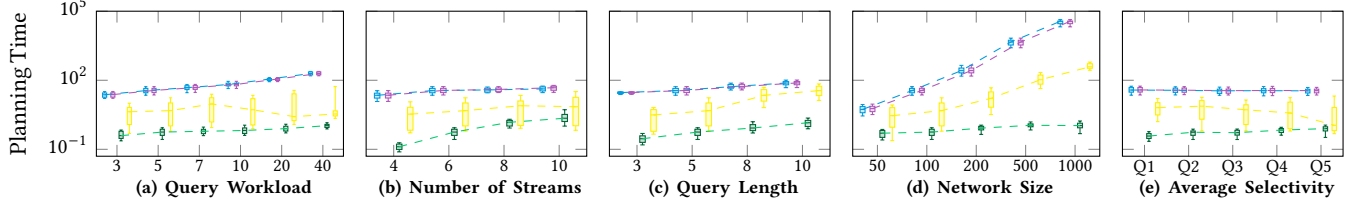


Figure 7: Comparison of KRAKEN against State-of-the-Art Approaches on Planning Time.

KRAKEN are only moderately affected, except for a similar sharp increase in hop latency for OP→PPC as the number of queries increases. Third, network size and event skew have only a minor effect on the median latency ratio (Figures 6d and 6e), influencing merely the spread of the distributions, especially for PrePP. The omitted plots for parent factor, node event ratio, and average selectivity preserve the same median ordering and show weak trends in our simulation.

Results. PrePP exhibits the highest hop-latency ratios, with a median of 5.0, because it assumes a centralized placement at the sink. Thus, all request/response messages must traverse from the cloud sink to (potentially deep) sources and back. Hence, latency accumulates over the required acquisition steps and grows with larger and longer workloads. In contrast, OP→PPC and KRAKEN leverage in-network placement, which shortens provider distances and can make frequently used inputs local, reducing both request and response path lengths. KRAKEN achieves a median hop-latency ratio of 2.33, improving over OP→PPC at 2.67 by 1.14 $\times$ (min. 1.03 $\times$, max. 2.33 $\times$). Finally, while increasing network size raises *absolute* hop latency because the topology depth grows, the *ratio* remains roughly stable since the all-push baseline scales similarly.

5.2.3 Planning Time Comparison. In this experiment, we explore the impact of different parameter settings on the planning time required by each planner to derive an execution plan.

Observations. We present our results in Figure 7, reporting planning time in seconds on a logarithmic scale. We make four observations. First, across all parameter settings, the median planning times follow a consistent ordering: INEv* $\geq$ OP→PPC $>$ KRAKEN $>$ PrePP. Second, INEv* and OP→PPC exhibit near-identical runtimes across all studies, indicating that the placement step dominates the sequential pipeline of OP→PPC. Third, KRAKEN shows a visibly larger runtime spread than the competing planners, reflecting stronger instance-to-instance variability in the number of feasible candidates explored during joint search. Fourth, for planners considering in-network placements (INEv*, OP→PPC, and KRAKEN), the planning

time exhibits exponential growth with network size (Figure 7d), consistent with the rapidly growing number of candidate placements. In contrast, PrePP is largely insensitive to network size but increases with the number of streams (Figure 7b), as its optimization operates over stream-level acquisition choices under a fixed sink placement. The omitted plots for parent factor, node event ratio, and event skew preserve the same median ordering. The node-event ratio and event skew exhibit only weak trends, whereas increasing the parent factor (denser connectivity) increases planning time for planners considering in-network placements by exposing more placement candidates.

Results. PrePP achieves the lowest median planning time (below 1 s; min. < 0.1 s, max. 9 s, avg. 0.8 s), because it optimizes acquisition under a fixed cloud placement and thus avoids enumerating placement candidates. Although the underlying push-pull optimization is NP-complete [36], the implemented greedy search and the reduced decision scope keep runtimes low. In contrast, planners that also optimize placement face an exponentially growing search space, consistent with the search-space growth discussed in Section 4.5. Despite its larger decision scope, KRAKEN achieves a median planning time below 5 s (min. < 0.1 s, max. 1,021 s, avg. 27 s). It outperforms INEv* (median 36 s, avg. 1,231 s, min. < 0.1 s, max. 58,968 s) and OP→PPC (median 36 s, avg. 1,233 s, min. 0.3 s, max. 58,969 s) in more than 90% of runs, by a factor of 7.2 $\times$ on the median. We attribute this counterintuitive result to four optimizations: (i) hierarchy-aware and availability-aware candidate node ordering, (ii) caching of acquisition results across repeated (*subquery, node*) evaluations, (iii) early pruning during expansion, and (iv) lightweight state generation. However, for large networks ($N \geq 500$), KRAKEN can exhibit maximum planning times above 1,000 s, although mean planning times remain lower: 122 s (± 75 s) for $N = 500$ and 440 s (± 167 s) for $N = 1000$. We discuss possible mitigation strategies for prolonged planning times in Section 4.6.

Note that we exclude the time for decomposition used by OP→PPC, INEv*, and KRAKEN, which is inherited from INEv but not used by PrePP. Including the time for decomposition (median 3.2 s, avg. 578 s, min. < 0.02 s, max. 446,615 s) would otherwise dominate planning time and affect all three placement-based approaches equally.

5.3 KRAKEN-specific Evaluation

In this section, we examine KRAKEN in isolation. We first analyze the impact of different search strategies in Section 5.3.1 and then study the effect of its bi-objective trade-offs in Section 5.3.2. Finally, we evaluate KRAKEN’s robustness under progressively tighter fog resource constraints in Section 5.3.3.

5.3.1 Impact of Search Strategy. In this experiment, we study how different search strategies affect both the solution quality and the planning time for our joint optimization problem. We compare three strategies: (1) *greedy*, which maintains a single current state and always selects the best successor, (2) *beam- k* , which maintains k states to explore a larger portion of the search space, and (3) DAG^* , which prioritizes partial plans by their estimated total cost. We evaluate beam search with $k = 3$ and $k = 5$. All strategies are configured to optimize only tuple volume ($\alpha = 1$) with varying topology and workload parameters, excluding topologies with more than 200 nodes.

Results. In terms of tuple volume, both beam search and DAG^* consistently outperform greedy search. In particular, beam-5 yields a lower tuple volume than greedy in 70.1% of the runs, and beam-3 in 65.1%. At the median, beam-5 reduces tuple volume by $\sim 10\%$ relative to greedy, while beam-3 reduces it by $\sim 7\%$, confirming that maintaining multiple partial plans helps the search to escape local greedy decisions. DAG^* further improves plan quality and reduces median tuple volume by $\sim 15\%$ relative to both beam- k variants. This shows that the admissible lower-bound heuristic provides additional guidance beyond maintaining a bounded set of promising candidates. By estimating the total costs, DAG^* can prioritize partial plans that are not immediately best, but lead to better complete plans. The similar performance of beam-3 and beam-5 suggests diminishing returns from wider beams: beam-5 outperforms beam-3 in only 55.8% of the runs, a substantially weaker advantage than either beam variant achieves over greedy. Thus, most of the benefit of beam search is already captured by a small beam width, whereas DAG^* provides a qualitatively different form of guidance through its lower-bound heuristic.

In terms of transmission latency, all strategies achieve the same median latency. This equivalence arises from the interaction between the hierarchical fog-cloud topology and our state-expansion order. The State Expander sorts candidate nodes by network distance from their sources, partitioning the topology into distance layers (nodes at equal distance). Thus, all strategies often place operators on nodes at the same distance to the sources, yielding identical path lengths. As a result, the search primarily explores horizontally within a layer rather than vertically across higher network tiers.

In terms of planning time, greedy remains the most efficient strategy. The overhead relative to greedy is $\sim 2.6\times$ for beam-3 and $4.2\times$ for beam-5, slightly below the naive linear scaling factors implied by the beam widths. DAG^* incurs a median overhead of $\sim 3\times$ compared to greedy, but exhibits a long-tailed planning-time distribution, increasing its mean planning time to $\sim 2.27\times$ that of beam-5.

Overall, DAG^* provides the best trade-off between cost and planning time by substantially lowering tuple volume compared to the other strategies while remaining competitive in median planning time. Beam-3 remains an attractive lightweight alternative when predictable planning time is more important than solution quality.

Table 2: Median Performance by α .

α	0.1	0.2	0.3	0.4	0.5	0.6	0.7	0.8	0.9
Latency Reduction (%)	50	50	46	39	33	16	11	8	1
Tuple Volume Increase (%)	88	84	85	47	40	4	1	1	0

5.3.2 Bi-Objective Trade-offs. In this experiment, we study how KRAKEN navigates the Pareto frontier between tuple volume and latency by varying the weight $\alpha \in \{0.1, \dots, 0.9\}$ in the scalarized scoring function $F(s)$. Tuple-volume increases and latency reductions are measured relative to the greedy baseline obtained under our default settings with volume-only optimization ($\alpha = 1.0$).

Observations. We summarize the volume-latency Pareto balance across all weight configurations in Table 2. We observe a smooth, monotonic relationship: configurations that reduce latency increase tuple volume, and vice versa, consistent with the Pareto frontier induced by our objective. A moderate setting of $\alpha = 0.3$ yields a favorable trade-off, achieving a 46% reduction in latency at an 85% increase in tuple volume. In contrast, aggressive latency prioritization with $\alpha = 0.1$ improves latency only marginally further (to a reduction of 50%) while increasing the tuple volume by 88%, indicating a limited benefit of pushing α lower. As α increases, the trade-off reverses. The crossover occurs near $\alpha = 0.6$, where a latency reduction of 16% incurs only 4% additional tuple volume. Beyond $\alpha = 0.7$, diminishing returns emerge: increasing α from 0.7 to 0.9 degrades the latency reduction by 8% while achieving almost no additional reduction of the tuple volume (from 1% to 0%).

Results. The smooth Pareto frontier indicates that KRAKEN’s bi-objective formulation, together with our min-max normalization, balances both metrics without constraint tuning (Section 4.3). Furthermore, even latency-leaning KRAKEN configurations reduce tuple volume relative to the sequential baseline $\text{OP} \rightarrow \text{PPC}$. For example, at $\alpha = 0.3$, KRAKEN reduces latency by 46% with a median tuple-volume ratio of 0.029 (versus 0.016 for $\alpha = 1.0$), which remains below the volume-greedy median of $\text{OP} \rightarrow \text{PPC}$ with a ratio of 0.033. Although these results are obtained under our default configuration and the breakpoints may shift with topology and workload parameters, the observed trends provide practical guidance for selecting α .

5.3.3 Impact of Resource Constraints. In this experiment, we study robustness under limited fog resources by progressively tightening per-node capacities over *resource constraint levels* $c \in \{0, \dots, 10\}$, where $c = 0$ denotes unconstrained fog nodes and each increment of c reduces the capacity budget of nodes at depth = 1 (one hop from the sink) by a factor $\sqrt{10}$, with capacities decreasing proportionally with depth. Since none of the baselines model resource feasibility, we compare KRAKEN only against PrePP, whose cloud-only placement remains feasible across all constraint levels. We use our default parameter settings, greedy search, and $\alpha = 1$ to match PrePP’s objective.

Observations. We present our results in Figure 8. Higher c values indicate stricter constraints, reducing the set of feasible in-network placements. We report *tuple volume* and *hop latency* ratios normalized to PrePP (black horizontal line), as well as the fraction of subqueries placed in the *cloud* and its complement, the *network* fraction. We make three observations. First, KRAKEN migrates placements toward the cloud as constraints tighten (min. 0.21, max. 0.99), while in-network placements decrease accordingly, as expected. The cloud share stays low under mild constraints ($c \leq 4$, avg. 0.31), increases

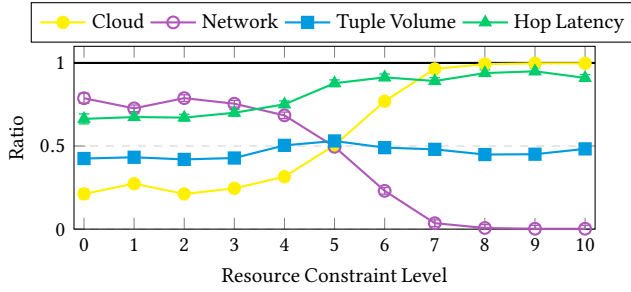


Figure 8: Effect of resource constraints on KRAKEN vs. PrePP.

at mid levels (avg. 0.63), and approaches almost full centralization for strong constraints ($c \geq 7$, avg. 0.99). Second, KRAKEN’s tuple volume advantage is largely stable across all levels, with an average ratio of ~ 0.46 . Third, the latency advantage gradually weakens as placements shift to the cloud, approaching PrePP once almost all subqueries are centralized (min. 0.66, avg. 0.81, max. 0.95).

Results. Overall, KRAKEN degrades gracefully under limited fog resources. Even when feasible, in-network placement becomes scarce, joint optimization preserves substantial tuple-volume savings over centralized push-pull acquisition. The sharp transition around $c = 5$ to cloud-centric placements reflects a workload-level trade-off between locality and in-network savings. As capacity constraints prevent co-locating multiple subqueries on the same fog node, KRAKEN has to place related subqueries on different nodes, causing shared inputs to be acquired and forwarded multiple times. Once constraints effectively centralize the workload, shared inputs become local again across subqueries, and redundant transfers largely vanish. In contrast, PrePP optimizes each query in isolation and may redundantly acquire the same input multiple times. The gradual loss of latency gains is consistent with longer provider distances as placements migrate to the cloud. Finally, planning time is on average $17.4\times$ slower than PrePP for mild constraints ($c \leq 4$), drops to $10.2\times$ at mid levels, and to $2.6\times$ for strong constraints ($c \geq 7$) as eligible fog nodes shrink.

6 Related Work

Efficient execution of continuous queries over distributed streams has been studied extensively in both stream processing and CEP [4, 17, 18]. Our work targets *joint* planning in reachability-constrained hierarchical fog-cloud topologies with heterogeneous node capacities and fixed sink(s) [45]. Many existing sequential planners assume homogeneous or mesh-like networks, which do not directly carry over to our hierarchical fog-cloud topology [7, 17, 25].

Operator placement. Operator placement decomposes queries into subqueries and assigns them to strategic nodes in the topology [33, 35], often combined with query reordering [7, 31, 47], prefix sharing, and merging for multi-query workloads [16, 32]. In CEP, INEv graphs [6], the successor to MuSE graphs [7], represent the state-of-the-art for in-network evaluation of multi-pattern CEP workloads. They refine decomposition beyond pattern hierarchies, introduce multi-node placement for stateful CEP operators, and model how intermediate results are forwarded to assemble full matches. However, these approaches are developed under assumptions that differ from our setting, e.g., homogeneous node capabilities

and flexible sink placement. In contrast, we target fog-cloud environments where low-end sources cannot host arbitrary operators [29] and fixed sinks act as global consumers. Accordingly, KRAKEN reuses INEv’s semantics-preserving decomposition and dependency order, but plans deployments under single-node placement with explicit feasibility constraints induced by the current partial plan.

For general-purpose stream processing in fog-cloud settings, several systems and planners consider heterogeneous resources and multiple objectives (e.g., fault tolerance or energy consumption), using single-node placement under all-push [1, 15, 28, 45]. Our work complements these efforts by making input acquisition a first-class planning decision and optimizing it jointly with placement.

Communication modes and joint optimization. Most distributed stream processing and CEP approaches rely on all-push communication, where operators continuously forward outputs downstream [4, 6, 7]. The push-pull communication scheme has been primarily studied in the context of CEP, which typically focuses more on stateful workloads than on general-purpose stream processing. Representative approaches include T-Rex [21], PECOP [23, 25], and Comet [17]. The state-of-the-art, PrePP [36], refines push-pull with *predicate-based* pulls that filter buffered events by content in addition to window constraints. However, most prior work follows the *sequential* pattern. To the best of our knowledge, PECOP [23] is the only approach that departs from this pattern by exhaustively optimizing placement and input acquisition jointly in homogeneous mesh networks. While this provides genuine joint optimization, exhaustive enumeration becomes impractical as topology size grows [23]. PECOP regains tractability through heuristics over predetermined operator configurations, but does not guarantee that a feasible execution plan is found. In contrast, KRAKEN avoids exhaustive enumeration by constructing plans incrementally according to a dependency order over subqueries. For each subquery, it evaluates candidate nodes together with concrete acquisition sequences and retains deployment decisions according to a search strategy. This enables tractable joint optimization for hierarchical fog-cloud deployments.

7 Conclusion

In this paper, we presented KRAKEN, an execution planner for hierarchical, heterogeneous fog-cloud topologies that *jointly* optimizes placement and input acquisition under compute and latency constraints. KRAKEN formulates planning as a state-space search over *atomic deployment decisions*, each binding a subquery to an execution node and an acquisition sequence, i.e., either all-push or push-pull. KRAKEN scores these decisions using a topology-aware ingress-egress formulation that rewards input locality and explicitly accounts for result dissemination, while enforcing feasibility by pruning candidates that violate per-node compute budgets or the global latency bound. Its decoupled architecture separates core planning components, enabling the evaluation of alternative component combinations. In a simulation study over different topologies and diverse workloads, KRAKEN consistently outperforms the sequential baseline, reducing tuple volume by about $3\times$ on average while improving hop latency by 14% and reducing median planning time by $7\times$. Overall, our results show that joint optimization, guided by a unified topology-aware cost model, yields robust improvements for resource-constrained hierarchical deployments.

Acknowledgments

We gratefully acknowledge funding from the German Federal Ministry of Education and Research under the grant BIFOLD26B. We thank Nur Ali Dilek, Riccardo Tommasini, Samira Akili, and our reviewers for their insightful suggestions and comments.

References

- [1] Ishfaq Ahammad. 2023. Fog Computing Complete Review: Concepts, Trends, Architectures, Technologies, Simulators, Security Issues, Applications, and Open Research Fields. *SN Comput. Sci.* 4, 6 (2023), 765. doi:10.1007/S42979-023-02235-9
- [2] Yanif Ahmad and Ugur Cetintemel. 2004. Networked Query Processing for Distributed Stream-Based Applications. In *(e)Proceedings of the Thirtieth International Conference on Very Large Data Bases, VLDB 2004, Toronto, Canada, August 31 - September 3 2004*, Mario A. Nascimento, M. Tamer Özsu, Donald Kossmann, Renée J. Miller, José A. Blakeley, and K. Bernhard Schiefer (Eds.). Morgan Kaufmann, 456–467. doi:10.1016/B978-012088469-8.50042-5
- [3] Adnan Akbar, Abdullah Khan, Francois Carrez, and Klaus Moessner. 2017. Predictive analytics for complex IoT data streams. *IEEE Internet of Things Journal* 4, 5 (2017), 1571–1582.
- [4] Mert Akdere, Ugur Cetintemel, et al. 2008. Plan-based complex event detection across distributed sources. *VLDB* 1, 1 (2008).
- [5] Tyler Akidau, Slava Chernyak, and Reuven Lax. 2018. *Streaming systems: the what, where, when, and how of large-scale data processing*. "O'Reilly Media, Inc".
- [6] Samira Akili, Steven Purtzel, and Matthias Weidlich. 2023. INEV: In-Network Evaluation for Event Stream Processing. *Proc. ACM Manag. Data* 1, 1 (2023).
- [7] Samira Akili and Matthias Weidlich. 2021. MuSE Graphs for Flexible Distribution of Event Stream Processing in Networks. In *SIGMOD '21: International Conference on Management of Data, Virtual Event, China, June 20-25, 2021*, Guoliang Li, Zhanhuai Li, Stratos Idreos, and Divesh Srivastava (Eds.). ACM, 10–22. doi:10.1145/3448016.3457318
- [8] Elias Alevizos, Anastasios Skarlatidis, et al. 2017. Probabilistic Complex Event Recognition: A Survey. *ACM Comput. Surv.* 50, 5, Article 71 (Sept. 2017).
- [9] Alexander Alexandrov, Rico Bergmann, et al. 2014. The Stratosphere platform for big data analytics. *VLDB J.* 23, 6 (2014).
- [10] Alexander Artikis, Alessandro Margara, Martin Ugarte, Stijn Vansummeren, and Matthias Weidlich. 2017. Complex Event Recognition Languages: Tutorial. In *Proceedings of the 11th ACM International Conference on Distributed and Event-Based Systems (Barcelona, Spain) (DEBS '17)*. Association for Computing Machinery, New York, NY, USA, 7–10. doi:10.1145/3093742.3095106
- [11] Lawrence Benson, Philipp M. Grulich, Steffen Zeuch, Volker Markl, and Tilmann Rabl. 2020. Disco: Efficient Distributed Window Aggregation. In *Proceedings of the 23rd International Conference on Extending Database Technology, EDBT 2020, Copenhagen, Denmark, March 30 - April 02, 2020*, Angela Bonifati, Yongluan Zhou, Marcos Antonio Vaz Salles, Alexander Böhm, Dan Olteanu, George H. L. Fletcher, Arijit Khan, and Bin Yang (Eds.). OpenProceedings.org, 423–426. doi:10.5441/002/EDBT.2020.48
- [12] Flavio Bonomi, Rodolfo A. Milito, Jiang Zhu, and Sateesh Addepalli. 2012. Fog computing and its role in the internet of things. In *Proceedings of the first edition of the MCC workshop on Mobile cloud computing, MCC@SIGCOMM 2012, Helsinki, Finland, August 17, 2012*, Mario Gerla and Dijiang Huang (Eds.). ACM, 13–16. doi:10.1145/2342509.2342513
- [13] Paris Carbone, Asterios Katsifodimos, Stephan Ewen, Volker Markl, Seif Haridi, and Kostas Tzoumas. 2015. Apache Flink™: Stream and Batch Processing in a Single Engine. *IEEE Data Eng. Bull.* 38, 4 (2015), 28–38. <http://sites.computer.org/debull/A15dec/p28.pdf>
- [14] Ankit Chaudhary, Kaustubh Beedkar, Jeyhun Karimov, Felix Lang, Steffen Zeuch, and Volker Markl. 2025. Incremental Stream Query Placement in Massively Distributed and Volatile Infrastructures. In *41st IEEE International Conference on Data Engineering, ICDE 2025, Hong Kong, May 19-23, 2025*. IEEE, 376–390. doi:10.1109/ICDE65448.2025.00035
- [15] Ankit Chaudhary, Steffen Zeuch, and Volker Markl. 2020. Governor: Operator Placement for a Unified Fog-Cloud Environment. In *Proceedings of the 23rd International Conference on Extending Database Technology, EDBT 2020, Copenhagen, Denmark, March 30 - April 02, 2020*, Angela Bonifati, Yongluan Zhou, Marcos Antonio Vaz Salles, Alexander Böhm, Dan Olteanu, George H. L. Fletcher, Arijit Khan, and Bin Yang (Eds.). OpenProceedings.org, 631–634. doi:10.5441/002/EDBT.2020.81
- [16] Ankit Chaudhary, Steffen Zeuch, Volker Markl, and Jeyhun Karimov. 2023. Incremental Stream Query Merging. In *EDBT 2023, Greece*.
- [17] Jianxia Chen, Lakshmi Ramaswamy, David K. Lowenthal, and Shivkumar Kalyanaram. 2012. Comet: Decentralized Complex Event Detection in Mobile Delay Tolerant Networks. In *13th IEEE International Conference on Mobile Data Management, MDM 2012, Bengaluru, India, July 23-26, 2012*, Karl Aberer, Anupam Joshi, Sougata Mukherjee, Dipanjan Chakraborty, Hua Lu, Nalini Venkatasubramanian, and Salil S. Kanhere (Eds.). IEEE Computer Society, 131–136. doi:10.1109/MDM.2012.18
- [18] Graham Cormode and Minos N. Garofalakis. 2005. Sketching Streams Through the Net: Distributed Approximate Query Tracking. In *Proceedings of the 31st International Conference on Very Large Data Bases, Trondheim, Norway, August 30 - September 2, 2005*, Klemens Böhm, Christian S. Jensen, Laura M. Haas, Martin L. Kersten, Per-Åke Larson, and Beng Chin Ooi (Eds.). ACM, 13–24. <http://www.vldb.org/archives/website/2005/program/paper/tue/p13-cormode.pdf>
- [19] Gianpaolo Cugola and Alessandro Margara. 2012. Complex event processing with T-REX. *Journal of Systems and Software* (2012).
- [20] Gianpaolo Cugola and Alessandro Margara. 2012. Processing flows of information: From data stream to complex event processing. *ACM Comput. Surv.* 44, 3 (2012), 15:1–15:62. doi:10.1145/2187671.2187677
- [21] Gianpaolo Cugola and Alessandro Margara. 2013. Deployment strategies for distributed complex event processing. *Computing* 95, 2 (2013).
- [22] Gianpaolo Cugola and Alessandro Margara. 2015. The Complex Event Processing Paradigm. In *Data Management in Pervasive Systems*, Francesco Colace, Massimo De Santo, Vincenzo Moscatò, Antonio Picariello, Fabio Alberto Schreiber, and Letizia Tanca (Eds.). Springer, 113–133. https://doi.org/10.1007/978-3-319-20062-0_6
- [23] Ioannis Flouris et al. 2020. Network-wide complex event processing over geographically distributed data sources. *Inf. Syst.* (2020).
- [24] Ioannis Flouris, N. Giatrakos, et al. 2017. Issues in complex event processing: Status and prospects in the big data era. *Journal of Systems and Software* (2017).
- [25] Ioannis Flouris, Vasiliki Manikaki, Nikos Giatrakos, et al. 2016. Ferari: A prototype for complex event processing over streaming multi-cloud platforms. In *SIGMOD. 2093–2096*.
- [26] Nikos Giatrakos, Elias Alevizos, et al. 2020. Complex event recognition in the Big Data era: a survey. *VLDB J.* (2020).
- [27] Paulo Jesus, Carlos Baquero, and Paulo Sérgio Almeida. 2014. A survey of distributed data aggregation algorithms. *IEEE Communications Surveys & Tutorials* 17, 1 (2014), 381–404. doi:10.1109/COMST.2014.2354398
- [28] Hyunhul Kang. 2013. In-Network Processing of Joins in Wireless Sensor Networks. *Sensors* 13, 3 (2013), 3358–3393.
- [29] Janine Kniess and Samuel Silva de Oliveira. 2020. Data reduction in sensor networks based on dispersion analysis. *Computing* 102, 5 (2020), 1159–1170. doi:10.1007/S00607-020-00795-9
- [30] Ilya Kolchinsky and Assaf Schuster. 2018. Efficient Adaptive Detection of Complex Event Patterns. *Proceedings of the VLDB Endowment* 11, 11 (2018).
- [31] Ilya Kolchinsky and Assaf Schuster. 2018. Join query optimization techniques for complex event processing applications. *VLDB* (2018).
- [32] Ilya Kolchinsky and Assaf Schuster. 2019. Real-Time Multi-Pattern Detection over Event Streams. In *SIGMOD. ACM*.
- [33] Geetika T. Lakshmanan, Ying Li, and Robert E. Strom. 2008. Placement Strategies for Internet-Scale Data Stream Systems. *IEEE Internet Comput.* 12, 6 (2008), 50–60. doi:10.1109/MIC.2008.129
- [34] Dan O’Keeffe, Theodoros Salonidis, and Peter R. Pietzuch. 2018. Frontier: Resilient Edge Processing for the Internet of Things. *Proc. VLDB Endow.* 11, 10 (2018), 1178–1191. doi:10.14778/3231751.3231767
- [35] Peter R. Pietzuch, Jonathan Ledlie, Jeffrey Shneidman, Mema Roussopoulos, Matt Welsh, and Margo I. Seltzer. 2006. Network-Aware Operator Placement for Stream-Processing Systems. In *Proceedings of the 22nd International Conference on Data Engineering, ICDE 2006, 3-8 April 2006, Atlanta, GA, USA*, Ling Liu, Andreas Reuter, Kyu-Young Whang, and Jianjun Zhang (Eds.). IEEE Computer Society, 49. doi:10.1109/ICDE.2006.105
- [36] Steven Purtzel, Samira Akili, et al. 2022. Predicate-Based Push-Pull Communication for Distributed CEP (DEBS). *ACM*.
- [37] Nicholas Poul Schultz-Møller, Matteo Migliavacca, and Peter Pietzuch. 2009. Distributed Complex Event Processing with Query Rewriting. In *Proceedings of the Third ACM International Conference on Distributed Event-Based Systems (Nashville, Tennessee) (DEBS '09)*. Association for Computing Machinery, New York, NY, USA, Article 4, 12 pages. doi:10.1145/1619258.1619264
- [38] Erricos Streviniotis, Dimitrios Banelas, Nikos Giatrakos, and Antonios Deligiannakis. 2025. DAG*: A Novel A*-Like Algorithm for Optimal Workflow Execution Across IoT Platforms. In *41st IEEE International Conference on Data Engineering, ICDE 2025, Hong Kong, May 19-23, 2025*. IEEE, 807–820. doi:10.1109/ICDE65448.2025.00066
- [39] Sriskandarajah Suhothayan, Kasun Gajasinghe, et al. 2011. Siddhi: A second look at complex event processing architectures. In *ACM workshop on Gateway computing environments*.
- [40] Juliane Verwiebe et al. 2022. Algorithms for Windowed Aggregations and Joins on Distributed Stream Processing Systems. *Datenbank-Spektrum* (2022).
- [41] Lei Ying, Zhen Liu, Don Towsley, and Cathy H Xia. 2008. Distributed operator placement and data caching in large-scale sensor networks. In *IEEE INFOCOM 2008-The 27th Conference on Computer Communications*. IEEE, 977–985.
- [42] Hai Yu, Ee-Peng Lim, and Jun Zhang. 2006. On In-network Synopsis Join Processing for Sensor Networks. In *7th International Conference on Mobile Data Management (MDM 2006), Nara, Japan, May 9-13, 2006*. IEEE Computer Society,

32. doi:10.1109/MDM.2006.113
- [43] Matei Zaharia, Reynold S Xin, Patrick Wendell, et al. 2016. Apache spark: a unified engine for big data processing. *ACM* (2016), 56–65.
 - [44] Steffen Zeuch et al. 2022. NebulaStream: Data Management for the Internet of Things. *Datenbank-Spektrum* (2022).
 - [45] Steffen Zeuch, Ankit Chaudhary, et al. 2020. The NebulaStream Platform for Data and Application Management in the Internet of Things. In *CIDR*.
 - [46] Haopeng Zhang, Yanlei Diao, and Neil Immerman. 2014. On Complexity and Optimization of Expensive Queries in Complex Event Processing. In *SIGMOD*.
 - [47] Ariane Ziehn et al. 2024. Bridging the Gap: Complex Event Processing on Stream Processing Systems. In *EDBT 2024, Italy*. OpenProceedings.org.
 - [48] Ariane Ziehn, Jan Szlang, Steffen Zeuch, and Volker Markl. 2025. Unraveling the Impact of Window Semantics: Optimizing Join Order for Efficient Stream Processing. *Proc. VLDB Endow.* 18, 8 (2025), 2468–2481. <https://www.vldb.org/pvldb/vol18/p2468-ziehn.pdf>